TheBench Guide

Cem Bozşahin|Ankara, İstanbul 2022–25
cem.bozsahin@gmail.com

For Linguistic Analysis, Description, and Typological Exploration and Fine-tuning of Natural Grammars with Two Command Relations

Version 2.2.2 July 26, 2026 coloured text means changes from the previous guide
Home: github.com/bozsahin/thebench (there are instructions here for install and use) Install

Contents

1 Introduction **1**

2 Syntactic command and semantic command **2**

3 Linguistics and mathematics behind the tool **4**

4 TheBench's organization **8**

5 Synthetics: The elements of grammar **9**

6 Analytics: Comparing the elements of grammar **11**

7 What is a model of grammar? **15**

8 Exploration: Case, agreement, grammatical relations, grammar modelling **17**

9 Recommended work cycles **21**

10 To do **22**

Appendix A: Mathematics of the r-command **25**

Appendix B: Mathematics of the t-command **26**

Appendix C: Sample uses of TheBench commands **27**

Glossary, references and the command syntax **33**

1 Introduction

TheBench is a tool to study linguistic structures intralinguistically and crosslinguistically through two command relations. It is for writing monadic grammars to explore analyses, to compare languages through their category collections to understand their connectedness and degree of variations (and its limits), to fine-tune models of grammar from the form-meaning pairs where syntax is a latent variable (i.e. it is unannotated, externally unsupervised).[1] A **monadic grammar** employs **function composition** as its only **invariant**. Monadic grammar

One particular monadic grammar, Bozşahin (2025), hereafter MG, for *Monadic Grammar*, which forms the basis of TheBench, does not try to explain away language by enumerating psychological primitives such as N, V, A, P, and a universal grammar, followed by lacunas and outliers. It tries to explain why, given *any* inventory of basic categories, all human languages attested or unattested would manifest a limited category space that is sufficiently rich and diverse to be called the locus of competence MG

[1] The child (or the hypothetical learner) may be guided for reference and attention, but in MG it is assumed to be not given any labels or ground truths about what is taking place. Sometimes this is called self-supervision, but the term must be taken with a grain of salt.

```
/\ Welcome to The Bench
/\ /\  A workbench for studying NL structures built by two command relations
/\ /\ /\ Bench version:       2.2.1 Dated: July 25, 2025
/\ /\ /\ Python version:      3.11.10
/\ /\ /\ Common Lisp version: SBCL 2.2.9
/\ /\ /\
/\ /\ /\ Pre/post pro
cessing by Python (grammar development, interfaces)
/\ /\ /\ Processing by Common Lisp     (analysis, training, ranking)
/\ /\ Today: August 05, 2025, 19:46:07
/\ Type x to exit, ? to get some help
```

```
lisp     : bench.lisp loaded, version 8.0, encoding UTF-8
         : bench.user.lisp loaded
python   : bench.py   loaded, version 2.2.1, encoding utf-8
Your working (current) directory is:
  /home/bozsahin/myrepos/thebench
ready
/\
```

Figure 1: The welcome screen of TheBench.

grammar and sufficiently small to be acquirable in a limited timeframe from limited exposure.

It is based on the idea that linguistic categories in the grammar are the only devices to decide the grammaticality of an expression *and* its consequent sense of meaningfulness so that self-supervision over form-reference pairs would be enough to make monadic grammars acquirable.[2] The *process* of analysis that TheBench models is the formal footprint to see the revealing (interpretation) and expression (production) of the two command relations. One such category landscape involving two particular command relations is offered by Bozşahin (2025). TheBench helps to model the category space of MG.

Figure 1 shows what you see when you launch TheBench. Software versions may vary. If you are interested in the software specification and the tool's use, please skip to §4. In the next two sections we look into a bit of linguistics and mathematics behind TheBench.

2 Syntactic command and semantic command

In MG, the need for two command relations, the need for categories as correspondences of them, arises from trying to account for the sense of meaningfulness which is consequent to the grammaticality of for example the following senseless expression (due to Chomsky). The idea is that both grammaticality and meaningfulness need explaining.

(1) Colorless green ideas sleep furiously.

Long story short, MG proposes that this meaningfulness is dependent on the well-formed syntactic command relations supporting certain semantic command relations, which are not obvious from the physical event itself but clear once the reference to the human reporting of the practice is established, for example being 'torpid' in this example, from Roman Jakobson. If we want to go further than saying something is grammatical or not, MG suggests that 'the **corresponding category**' is a forced move.

If we want to also explain under what conditions the following expression might be grammatical,

(2) *She played the piano in an hour

Grammaticality & meaningfulness

MG proposes that it would be because of a reference shift to another category by which the speaker-signer refers to an achievement or accomplishment (e.g. a genius mastering an instrument in a very short period), two human-specific social conceptions and

[2] For the choice of term form-reference pairing over form-meaning pairing, see §7.

reporting of events over and above the physical properties of the events themselves, which, via their own semantic command relations, switches to another syntactic command relation to make it grammatical in the first place. Meaningfulness is always a consequence of grammaticality.

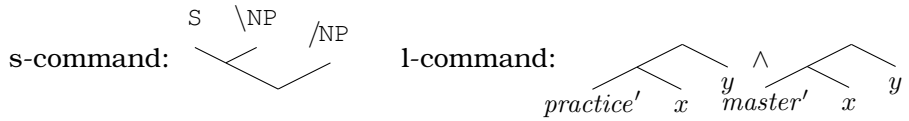
MG shows that the category landscape of command correspondences goes to considerable depth and breadth in understanding intralinguistic and crosslinguistic diversity in structural functions of case, agreement and grammatical relations. There are tools in TheBench to study these structural functions. It is claimed that it is not just idiosyncrasy that examples such as (2) point out; the problem addresses is the universal properties of linguistic structure, for example case, agreement, and grammatical relations; see e.g. (7–8) for a start. Universals

The two command relations are the syntactic command and the semantic command, respectively called the s-command and l-command in MG. 'A s-commands B' means A compares later than B syntactically (e.g. comparing two arguments of a verb).[3] A asymmetrically bears more syntactic information than B in comparison because A can 'see what happens to' B but B cannot see what happens to A. s-command and l-command

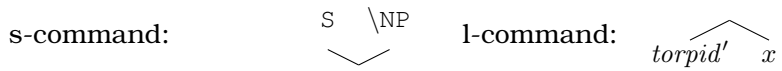
'A l-commands B' means A compares later than B semantically (e.g. comparing the differences in the prominence of the arguments of a verb or their placeholding in meaning). A asymmetrically bears more information of that kind than B. They are put in a correspondence for every element of a monadic grammmar in the form of a *category*.

For the two examples above, the idea of categories as correspondences can be thought of as follows, denoted with ':'.

(3) a. **played** :: (S\NP)/NP : $\lambda x \lambda y.practice'xy \wedge master'xy$



b. **sleep** :: S\NP : $\lambda x.torpid'x$



We show the s-command and l-command as asymmetrically structured elements subsequently for (a) and (b). Notice that *this* 'play' refers to mastery by practice, with subsequent placeholders for it, 'sleep' to torpid, its x referring to say a thing with some kind of uneasiness. In (a), the \NP s-commands the /NP in giving S. In its l-command, the y's l-command the x's.

The monadic structures are binary comparison of elements that employ semantics of composition only, in a hermetic seal, hence the name; see Mac Lane (1971); Moggi (1988) for that. Comparison of objects (the elements of grammar) is done with 'arrows' in category theory, which we can think of as functions. Analysis then is the structuring of comparison. This structure is the basis of interpreting an expression, or, building one. Either can be done using the top-down or bottum-up parsing techniques. The structure reflects the command relations (production) or takes an expression to the objects of grammar (interpretation). **The unfolding of the semantic command relation is consequent to the unfolding of the syntactic command relation.** They are Monadic structures

Syntactic autonomy & semantics

[3] 'Compare' is the term used by category theory for analyzing all kinds of objects that we do not wish to take apart because they are assumed to be already put together—synthetic. They are presumed to be related because otherwise we wouldn't categorize them together in the first place. Therefore abstraction (as functions, 'arrows') is the only way to compare them. Binary composition is the only basis of their comparison. Comparison is therefore inherently structured because 'later than' is an abstract asymmetric relation; we can compare one at a time in binary composition. The objects in the case of natural languages would be the elements of grammar. They are visible to comparison only through their categories. Interpretation and production are both based on comparison. The first process compares an expression in the analysis to pick the elements of grammar than can support it. The second process compares the elements of grammar to form an expression in the analysis that is a reflection of them. Therefore analysis can mean 'untie' and 'through combination.'

free to vary within themselves. All variations have been attested in some language of the world according to MG.

TheBench is essentially old-school categorial grammar to represent the idea of correspondences, with the implication that although syntax is autonomous (recall *colorless green ideas sleep furiously*), the treasure is in the baggage it carries at every step, viz. semantics, more narrowly, the predicate-argument structures indicating choice of categorial reference and its consequent placeholders for decision in such structures.

There are some new thoughts and gadgets that are brought to the old school in TheBench. Unlike traditional categorial grammars, application is turned into composition in the monadic analysis. Every correspondence requires specifying the two command relations, one on syntactic command and the other on semantic command. A monadic grammar of TheBench contains only synthetic elements (called 'objects' in category theory of mathematics) that are shaped by this analytic invariant, viz. composition. Both ingredients (command relations) of any analytic step must therefore be functions ('arrows' in category theory).

Categorial grammar and MG novelties

Having to have functions requires a bit of explaining on the linguistic side, which I do in the next section.

3 Linguistics and mathematics behind the tool

Mathematically, a monad composes two functions f and g as the only basis of object comparison to maintain the dependency of f on g, which we can write in extensional terms as $\lambda x.f(gx)$. Notice that this is asymmetric: g does not depend on f, but f depends on g.

It does so by doing $g \circ f$. Computationally, it first gets g, then f. The ultimate element f is always the head function; see Gallier 2011:118 for reasons for the notation. In this representation, $g \circ f$, it is clearer that f depends on g in $\lambda x.f(gx)$, not the other way around. In the alternative notation, writing instead $f \circ g$ to mean $\lambda x.f(gx)$, and saying that $f \circ g$ means 'do g, then f' to get the dependency right, is somewhat counter-intutitive, especially if we are going to construct all structure from sequencing and reference categorization by a single chain of linguistic dependency. Note that $\lambda x.f(gx)$ dependency is a single chain: g depends only on x, and f depends only on (gx). The linguistic significance of these properties is that, because the monad always returns the result of the ultimate element, the monad itself is oblivious to the heads and their directionality. The heads themselves must specify that information, one at a time. MG has a lot more to say about that.

The command monad of TheBench internally turns f and g into semantic functions even if one of them is syntactically not a function. (If none of them is a *syntactic* function, they would not be comparable, with implications—in my opinion—for morphological compounding.) It does so by deriving the case functions from the verbs and verb-like elements of a grammar, and by keeping intact the two command relations, which every element of grammar specifies: surface command (s-command) and predicate-argument command (l-command).

The s-command specifies in what order and directionality an element takes its syntactic arguments. It is asymmetrically structural; it is not a linear metric. It embodies the syntactic notion of combining later or earlier than other elements. The l-command specifies in what order and dominance the predicate-argument structure is formed. It is similarly asymmetrically structural. Pairing of the two is always required in the monadic grammar of TheBench.

Command pairing

This pairing, in the form of a *procedural category*, is a concept we can think of as a culmination of all of the following in one representational package for singular comparison: Montague's (1973) basic rules associating every syntactic application with logical interpretation, Bach's (1976) rule-to-rule thesis, Klein and Sag's (1985) one-way translation of syntactic and semantic types, Grimshaw's (1990) a-structure, and Williams's (1994) argument complex, without nonlocality, that is, without specification of argu-

Procedural categories

ment's arguments—maintaining his locality of predication, Hale and Keyser's (2002) syntactic configuration of a lexical item and its semantic components, and Abelson and Sussman's (1985) procedural epistemology.

A procedural category has the added role of choice of reference *for the whole* reflected in its syntactic type to lay out decision points for truth conditions of the parts in the semantic type. The main implication for the grammarian is that, perforce, there is a need for language-variant linguistic vocabulary to explore case, agreement, grammatical relations and other structural functions given such invariant analytics, because only the variant vocabulary and preserved command relations seem to be able to make *any grammar* transparent and consistently interpretable with respect to the invariant (composition). Insisting on the transparency has dividends for the modelling and exploration of grammar acquisition.

Grammars & models

Syntactic, phonological, morphological and predicate-argument structural reflex of composition-only analytics is the main practice of studying linguistic structures with a monad. `TheBench` is a tool implementing the idea. To sum up, it is an old-school categorial grammar, except that it uses composition only. That is, application is also turned into composition, and, multiple dependency projection in one step (such as CCG's **S** projection) is not analytic; it has to be specified by the head of a construction.

Case functions

Turning application into composition has implications for case, consequently for agreement and grammatical relations. It works as follows. Consider the following analysis in the context of applicative categorial grammar, much like Ajdukiewicz (1935); Bar-Hillel (1953); Montague (1973):

(4)
$$\overset{\mathtt{S}}{:admire'\,john'\,sincerity'}$$

$$\overset{\mathtt{NP_{3s}}}{:sincerity'}\ \textbf{Sincerity} \qquad \overset{\mathtt{S\backslash NP_{3s}}}{:\lambda y.admire'\,john'\,y}$$

$$\overset{\mathtt{(S\backslash NP_{3s})/NP}}{:\lambda x\lambda y.admire'\,x\,y}\ \textbf{admires} \qquad \overset{\mathtt{NP}}{:john'}\ \textbf{John}$$

Here, the verb applies to its arguments one at a time, first to its syntactic object, then to its subject. Application is evident on the semantic side, written after ':'.

Inside the monad

What takes places inside the monad is the following, where both applications are via function composition, in the template of $\lambda z.2(1\,z)$; and, the 2 function, the one that gives the result, is always the ultimate element (it is underlined at every step):

(5)
$$\overset{\mathtt{S}}{:admire'\,john'\,sincerity' = \lambda z.[\underline{\lambda\psi.\psi(\lambda y.admire'\,john'\,y)}][(\lambda\phi.\phi\,sincerity')z]}$$

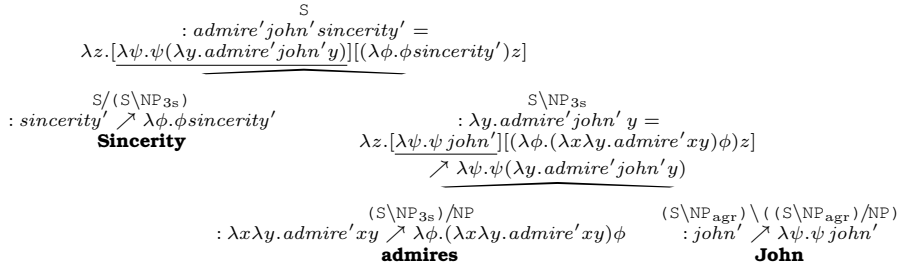
$$\overset{\mathtt{NP_{3s}}}{:sincerity' \nearrow \lambda\phi.\phi\,sincerity'}\ \textbf{Sincerity} \qquad \overset{\mathtt{S\backslash NP_{3s}}}{:\lambda y.admire'\,john'\,y = \lambda z.[\underline{\lambda\psi.\psi\,john'}][(\lambda\phi.(\lambda x\lambda y.admire'\,x\,y)\phi)z] \nearrow \lambda\psi.\psi(\lambda y.admire'\,john'\,y)}$$

$$\overset{\mathtt{(S\backslash NP_{3s})/NP}}{:\lambda x\lambda y.admire'\,x\,y \nearrow \lambda\phi.(\lambda x\lambda y.admire'\,x\,y)\phi}\ \textbf{admires} \qquad \overset{\mathtt{NP}}{:john' \nearrow \lambda\psi.\psi\,john'}\ \textbf{John}$$

A monad always composes this way.

The implication for linguistic theory is that, the semantic lifting that takes place to engender composition, which is represented in the example using $\nearrow$, must be accompanied by a lifted syntactic category, for all elements. For the arguments of the verbs, these are the case functions, for any kind of argument including the clausal arguments of *think*-like verbs. Category theory suggests that case is a natural transformation which keeps the comparison commuting, in effect supplying the theoretical basis for the following empirically motivated case functions for the same example—empirical because they are all derived from the verbal subcategorizations in a grammar.

(6)



The narrowing of reference can be observed from the case functions, for example for agreement. The subject agreement reduces the options for reference from S/(S\NP) to S/(S\NP$_{3s}$) for third-person singular subject, which can be read off the verbal subcategorization if distinct verb forms make the distinction in their categories. For example, the following verb forms can bear the distinct but related categories—notice their l-commands— to account for the grammaticality and ungrammaticality by case functions alone:

Case & category

(7) a. *studies* :: S\NP$_{3s}$: $\lambda x.study' x$ (finite form)
b. *to study* :: VP : $\lambda x.study' x$ (stem form)
c. *study* :: IV : $\lambda x.study' x$ (root form)

(8) a. Mary /studies/*study/*to study.
b. Mary wants /to study/*studies/*study.
c. Mary may /study/*to study/*studies.

Such case functions refer to different kinds of events of the same predicate, (a) to the actual or imaginary worlds in which Mary studies, Montague-style, (b) to the choice of potential worlds where Mary could be considered to study, which can be assumed to be constructed by the root forms, (c). The ungrammaticality above therefore could be the result of event-referential mismatch. For categories to do all the linguistic work on (un)grammaticality, such distinctions must be categorized. We can mark these three different semantic actions (refer, potentially refer, construct to refer) in the l-commands as well; see MG, chapter 5.

Cross-categorial generalizations are possible, also from verbal subcategorization, for example grammatical relations and systems of accusativity and ergativity. Such categorizations are not external to grammar. For example the Dyirbal control verb subcategorizes for a VP$_{abs}$, meaning a VP with implicit absolutive argument. In an ergative language, it would be the single argument of the intransitive and the more patient-like argument of the transitive, which is the hallmark of ergative 'alignment' . It is captured categorially without linking theories of grammatical roles as primitives.

Notice that in (6) the verb itself is also 'lifted' semantically inside. Although, formally speaking, its lifted type is eta-equivalent to its unlifted type, the implication is that the lifted type is a function over and above who does what to whom. For the verb itself, and for similarly complex categories, their lifting can be associated empirically with spatiotemporality. Since ϕ in the example is a transparent function arising from analytics, tense and aspect must be language-particular functions over verbal types (see Klein 1994; Klein, Li and Hendriks 2000). For the arguments of the verbs, the lifting of their basic syntactic type relates to case—because they are subcategorized for. Such functions are crosslinguistically inferrable in TheBench from verbal subcategorizations; NB. the 'c command'.

For case, all cases are structural (i.e. second-order) functions in MG because they can be read off, **synthesized**, from the first-order functions such as the verb. Identifying different subcategorizations of distinct verb forms (even for the same verb, as its root, stem and finite subcategorizations) leads to a bottom-up theory of 'universals', that is, categorizations available to everyone, from the ground up.

Synthetic case

Unlike generative grammar, there are no grammar-external devices for studying grammaticality such as EPP, minimal link condition (MLC) or universal lexical categories such as N, V, A, P, or universal argument types such as those in X-bar theory. All grammaticality and ungrammaticality are decided by the local categories in a

grammar.The invariant circumscribes what they can be. The Husserlian and Polish-school idea is there, that segments arise because analysis-by-categories seeks out reference fragments in an expression. We do not assume that analysis arises from (or rules 'generate' from) Harris-style distributional segments.

Grammaticality, intensions, distributions

TheBench is meant to explore configurationality, narrowly understood here as studying the limits on surface distributional structure, *pace* Hale (1983),[4] and referentiality, together. TheBench's configurational approach to surface structure is inspired by Lambek (1958); Steedman (2020). Its referential approach to constructing elements is inspired by Sapir (1924/1949); Swadesh (1938); Montague (1970, 1973); Schmerling (2018). The following aspects can help situate the proposal in the landscape of theories of grammar.

Inspirations

Formally, as stated earlier, unlike categorial grammars such as that of Steedman (2000), all analytic structures handle one dependency at a time. There is no **S**-style analytic structure taking care of multiple dependencies at once. Unlike Montague grammar, predicate-argument structures are not post-revealed (*de re, de dicto*, scope inversion, etc.), because different categorization due to different reference is expected to play the key role in the explanation.[5] Moreover, the syntactic types of the second-order (structural) functions including those for case, agreement and grammatical relations require a linguistic theory precisely because they are all based on grounded (first order) elements.

Distinct function-argument and argument-function sequencing of reference is the cause of categorial and structural asymmetries in the monadic grammar of TheBench, which is an idea that goes back to Schönfinkel (1920/1924). He had used the idea to motivate his combinators, all but three (binary **B**, **A** and **T**) considered to be too powerful in TheBench for monadic structures. In their simplest forms they are:

(9) a. $\mathtt{X/Y}\ \ \mathtt{Y} \Rightarrow \mathtt{X}$ **(A)**
b. $\mathtt{Y}\ \ \mathtt{X\backslash Y} \Rightarrow \mathtt{X}$ **(T)**
c. $\mathtt{X/Y}\ \ \mathtt{Y/Z} \Rightarrow \mathtt{X/Z}$ (forward **B**)
d. $\mathtt{Y\backslash Z}\ \ \mathtt{X\backslash Y} \Rightarrow \mathtt{X\backslash Z}$ (backward **B**)

Steedman and Baldridge (2011) provide more information about them. When successful in comparison, they carry semantics with them, driven by syntactic category, because they are in fact doing the following:

(10) a. $\mathtt{X/Y}: f \quad \mathtt{Y}: a \Rightarrow \mathtt{X}: fa$ (C1)
b. $\mathtt{Y}: a \quad \mathtt{X\backslash Y}: f \Rightarrow \mathtt{X}: fa$ (C2)
c. $\mathtt{X/Y}: f \quad \mathtt{Y/Z}: g \Rightarrow \mathtt{X/Z}: \lambda x.f(gx)$ (C3)
d. $\mathtt{Y\backslash Z}: g \quad \mathtt{X\backslash Y}: f \Rightarrow \mathtt{X\backslash Z}: \lambda x.f(gx)$ (C4)

The names refer to 'comparison'. Bozşahin (2025, Appendix A; (4)) explains how C1, C2 are turned into composition, and C3, C4 handle composition over two- or more-argument functions, not just one. You will see the names in (10) in displays of TheBench analyses.

Analysis by comparison

What's left behind by disallowing multiple dependency passing, viz. composition, including internal composition with **A** and **T**, is anybody's composition, but with some exploratory power arising from function-argument and argument-function distinction because it allows transparent transmission of reference. We don't really know whether category choice by the speaker-hearer-signer is compositionally determined. We would be studying how far it can be transmitted compositionally in syntax starting with the whole, but not holistically. (This sounds like a distant memory in psychology too; see Koffka 1936).

One dependency at a time

[4] From this perspective, there is no such thing as a non-configurational language in the sense of Hale (1983).

[5] Cf. *Every professor wrote a book* where we can conceive one and the same book written by all and *Every student missed a meal*. Not all predicates allow for inversion, therefore it is not a far-fetched idea to replace post-reveal of scope with differences in the referentiality of the predicate and consequent change in type-raising of the arguments. In other words, neither direct compositionality of Montague (1970) nor quantifying-in of Montague (1973) are suggested as alternatives, relying instead on the verb's referential properties and the category of the quantifiers and non-quantifiers. TheBench is more compatible with the 1970 idea, but arguing instead to support different subcategorization due to different referentiality.

Empirically, morphology and syntax do not compete for the same task in monadic analysis, for example for surface bracketing. Morphology constructs and syntax transmits reference. Morphology is not confined to the 'lexicon', to 'operators' or to some other component, or to leaves of a tree. It is an autonomous structure, obviously not isomorphic to phonological or syntactic structure but also not subserving them either, projected altogether homomorphically in analytic structure. Morphology is considered to be the category constructor for the form, therefore all languages have morphology. And, referential differences in all kinds of arguments cause categorial differences including those in the idioms and phrasal verbs (Bozşahin, 2023); see §7 for more on the implications of these aspects for modelling.

Morphology does not compete with syntax

Typologically, any language-particular difference in elementary (i.e. synthetic) vocabulary can make its way into invariant analytic structure presuming it is the empirical motive to make argument-taking transparent. MG is a very verb-centric view of grammar. The elements' morphology plays a crucial role in transparency of analytics. This is one reason we avoid commitment to certain ways of doing morphology in TheBench.

The verb

4 TheBench's organization

The only user executable for TheBench is called thebench.[6] Install instructions make it accessible from any directory. TheBench is containerized to isolate itself from your system. When launched, it will display the welcome screen in Figure 1. All the files of TheBench are text files. There are two kinds: editable files and uneditable files. The editable files are grammar files, those created by the user using some plain text editor, and those generated by the system for the benefit of the user. The latter kind can be cut and pasted to plain-text grammars. A list of commonly-used operations can be saved as macro files to avoid redundant typing. These too are plain text editable files. The location of all editable files is your working directory. The uneditable files are not saved in your working directory to avoid clutter. They are in /var/tmp/thebench.

The App

Editable file

File locations

Technically, the editing functions of TheBench, the functions that transform internal files to editable files, and the interfaces of TheBench, are all written in Python. The processor functions are written in COMMON LISP. The installer (NB. the top of the first page) takes care of the software requirements to work with all popular personal computer platforms (well, almost all). The command interface, which is given at the back, has its own syntax combining the two aspects for editing and processing, which you can use online and offline (the '@ command', aka. 'batch mode'—see the basic glossary at the back).

Python & Lisp

There is no need to know either COMMON LISP or Python to use TheBench. Maybe this much is good to know to understand the processor's output: T means true, and NIL means false in all Lisps. You will encounter them quite often in using the interface. Use the '? command' when you launch TheBench to recall the full list of commands.

The processor's internal files go into the directory created by TheBench installer for you; they reside in /var/tmp/thebench. Occasional clean-up of your working directory and this directory is recommended.[7] Transformation of an internal file to an editable grammar is possible, which takes a file from /var/tmp/thebench to save the editable file in your working directory.

Internal files

The start-up routine and the training command 't' load an extra processor file called bench.user.lisp which is available in the repository. You can change that file, but please do not rename, relocate, disrupt or delete it. If it is not loadable, the system would complain and exit. No internal code depends on the code contained in this file. It is useful for experimenting on model parameters without changing TheBench code if you are going to do modelling.

[6] For reasons of transparent security, we mention the other executable: bench.train.sh, which is called from within TheBench. It operates offline for training sessions, which can last quite long.

[7] See the end of §9 for when not to do a clean-up.

Direct IPA support is too unwieldy for the tool, see to-do list. It seemed best to use UTF-8 tools for it in Python and Lisp. When you launch TheBench, please check the encoding reported by the processor. Both Python and COMMON LISP in your computer must report UTF-8, as in Figure 1.

IPA and UTF8

5 Synthetics: The elements of grammar

There are three kinds of elements of grammar in TheBench:

Entry types

(11) a. an elementary form bearing a functional category,
b. an asymmetric relational category relating the two categories of one form, and
c. a symmetric relational category relating the categories of two forms.

Examples in TheBench notation are respectively:

TheBench notation

(12)
```
likes  | v :: (s\^np[agr=3s])/^np : \x\y.like x y     % ^: object can topicalize
  #np-raise np[agr=?x] : lf  --> s/(s\np[agr=?x]) : \lf\p. p lf
  #tense runs, s[t=pres,agr=3s]\np:\x.pres run x <--> ran, s[t=past]\np:\x.past run x
```

Every entry must start in a new line and it must be on one line, without line breaks. This allows us to minimize non-substantive punctuation. Let the longer lines wrap around on the screen. Everything starting with '%' until the end of a line is considered to be a comment. Capitalization is significant only in phonological elements (the material up to the '|' for entries of type (a), the stuff separated from the category by the comma in type (c).).

Always start and stay on newline. % comments

Whitespacing is not important anywhere. Empty and comment-only lines are fine.

Whitespace

The relative order of specification of elementary items and symmetric relational categories in the grammar's text file is not important. Asymmetric relational categories apply in the order they are specified. TheBench home repository contains sample grammars as a cheat sheet. (My personal convention is to put the commands that process or generate TheBench commands in '.tbc' files, for 'the bench command'. The '@ command' can take them as macros to run.)

Order of elements not important except relational rules

The first kind of element in (12) starts with space-separated sequence of UTF-8 text, ending with '|'. Multi-word entries, non-words, parts of words, punctuation and diacritics are possible. (This is provided so that we can avoid single or double-quoted strings at any cost; there is no universal programming practice about strings, and the concept is overloaded.)

The form & MWEs

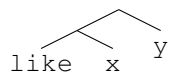
The material before '|' is considered to be the textual proxy of the phonological material of the element. The next piece is the part of speech for the element, which can be any token (see basic glossary at the back). The material after '::' is the category of the element. (This token is the most common convention in monads; it is called the monadic type constructor.) The part before the single ':' is called the syntactic type, which is the domain of s-command. The part after ':' is called the predicate-argument structure, which is the domain of l-command.

Tokens

Left-associativity is assumed for the bodies of both s-command and l-command. Right-associativity is assumed for lambda bindings. Together they constitute a syntax-semantics correspondence for the element. If you don't like such associativity and want to write the structure yourself, please be careful with the proliferating numbers of parentheses. The system will warn you about mismatches, but the process of getting them right can be painful. In training and exploration with complex elements it's easy to make judgment errors in parenthesization. With associativity, we can see that for example the l-command of the first entry in (12) is equivalent to (like x)y, in asymmetric tree notation as follows. Its lambda binders are not part of the command relation.

left associative s-command, l-command

l-command



The correspondence is explicit in the order of syntactic slashes and lamba bindings. For the first example in (12), '\x' in the lambda term corresponds to '/^np', and '\y'

Correspondence

to '\^np[agr=3s]'. The '\' in a predicate-argument structure is for 'lambda'. Several lambdas can be grouped to write a single '.' before the start of a lambda body, as we did in the example. We could also write them individually, as '\x.\y.like x y'.

Basic category

The syntactic type can be a (i) basic or (ii) complex category. (The term 'category' is quite commonly used also for the 'syntactic type' when no confusion arises.) A basic category is one without a slash, for example 'np[agr=3s]'. Here, 'np' is the basic category and '[agr=3s]' is its feature. Features are optional; they can be associated with the basic categories only (unlike many other phrase-structure theories). Multiple features are comma-separated. An unknown value for a feature is prefixed with '?'. For example 'np[person=1,number=sing,agr=?a]' means the agreement feature is underspecified.

Features

Complex category

The complex categories must have a slash, e.g. 's\^np[agr=3s]' in the example. It is a syntactic function onto 's' from '\^np[agr=3s]'. The result is always written first, so for example 's/np[agr=3s]' would be a syntactic function onto 's' from '/^np[agr=3s]'. Directionality would be different. In the first case, the function would look to the left, in the latter, to the right. In (s\^np[agr=3s])/^np of the first entry in (12), there is the s-command relation because there are two arguments; see (3a) for why the /^np is s-commanded by the \^np[agr=3s].

s-command

Slash modality

Modal control on the directionality is optionally written right after the syntactic slash, such as '^' in the first entry. If omitted, it is assumed to be the most liberal, that is '.'. Slash modalities are from Baldridge (2002); Steedman and Baldridge (2011). TheBench symbols for the modalities are

(13) a. . for '.',
b. ^ for '◇',
c. ⋆ for '⋆', and
d. + for '×'.

They are for surface-syntactic control of composition in the monad. The '.' modality in a category means any comparison rule can be applied to the bearer of this category. The '^' modality means only harmonic composition can apply, e.g. X/Y Y/Z, or Y\Z X\Y, where the slashes are always in the same direction. The '⋆' modality means only application can compare. The '+' modality means composition can compare crossed slashes such as X/Y Y\Z and Y/Z X\Y. They are about access of the category of an element to comparison rules in the invariant. They do affect the number of analyses produced. Please keep this in mind when writing categories.

Slash & number of analyses

Two more tools of TheBench are unique to MG. They are not in every categorial grammar's toolbox. There are the double slashes '\\' and '//' which are similarly backward and forward. They take and yield potential elements of grammar. (Therefore they are unlike Montague's multiple slashes, and they are not necessarily morphological.) They are called *extending categories* in MG, because of potentially extending the object set of a grammar. There is no modal control on them.

Double slashes

Another unique feature of MG is the distinction in basic categories. Basic categories can be singletons, that is, they can stand for their own value only. For example, (S\NP)/'the bucket' is a bonafide MG category. It has the singleton 'the bucket' which stands *as category* for the phonological sequence the bucket; cf. NP, which stands for many expressions that are distributionally NPs. Singletons can be single- or double-quoted as long as they are atomic (i.e. two successive single quotes do not make a double quote). Singletons are applicative because they cannot be the range of a complex category, they are domains only ; see Bozşahin (2023).

Singleton categories

In summary, elementary items bear categories that are functions of their phonological form.

The second kind of element in (12) is an asymmetric relational category. It means that the element bearing the category on the left of '-->' at surface form *also* has the category on the right. The token immediately after '#' is the name of the relation, in this case 'np-raise'. For such relations to make sense, the first lambda binding on the right must be on the whole predicate-argument structure on the left. This is not checked by the system, we can write any category in principle; but, this is why such

relations are not associated with a particular substantive element. Notice that there is no phonological element associated with the relation.

The third kind of element in (12) is a symmetric relational category. It means that during analysis either form is eligible for consideration along with its particular category. The analyst apparently considered them to be grammatically related so that they are not independently listed. (This is one way to capture morphological paradigms but not the only way.) The relation's name is right after '#'. The material on the left and right of '<-->' must also contain some phonological material (whitespace-separated words ending with a comma) so that we can reflect the substantive adjustment on the categories.

There are no elements, functions or relations in the monadic analysis which can alter or delete material in surface structure because that would not be always semantically composable. (It follows that IA/IP/WP morphology of Hockett must be sprinkled across the three kinds of specification above. Judging from the fact that no morphologically involved language is exclusively IA, IP or WP, this should not be surprising.)

Empty categories are not comparable; maybe empty elements are

No element type can depend on or produce categorially empty elements, because such elements aren't categories. (Phonologically empty categories would not be empty categories in MG; see §7 for more.) The monad's hermetic seal, that every analytic step is atomic, is also consistent with these properties. Together they allow us to adhere to the Schönfinkelian idea of building hirerarchical structures from sequential *asymmetries* alone.

6 Analytics: Comparing the elements of grammar

The tripartite organization of every synthetic element is reflected in TheBench notation:

(14) `phonology :: s-command : l-command`

It is transmitted unchanged in analytics. Self-distribution of morphology as it sees fit in a grammar is based on the common understanding that syntax, phonology, morphology, semantics, and information structure are obviously not isomorphic, but, equally obviously, related, in constructing and transmitting reference.

The analytic structure is invariant. This means that these domains are homomorphic to analytic structure, because there is only one such structure. It is function composition on the semantic side. How the forms (syntax, phonology and morphology) cope with that structure and limit the others is the business of a linguistic theory.

Information transmission in analytics therefore needs a closer look. In building structures out of elements one at a time, the method of matching basic and complex categories of the s-command is the following. If we have the two-category sequence (a) below, we get (b) by composition, not (c); cf. `f2` passing.

Term unification

(15) a. `s[f1=?x1,f2=v2]/s[f1=?x1]  s[f1=v1,f2=?x2]/np[f2=?x2]`
b. `s[f1=v1,f2=v2]/np[f2=?x2]`
c. `s[f1=v1,f2=v2]/np[f2=v2]`

This avoids unwanted generation of pseudo-global feature variables, and keeps every step of the comparison local. Using this property we could in principle compile out all finite feature values for any basic category in a grammar and get rid of its features (but nobody did that, not even GPSG. Maybe encoder-decoder transformer nets can make a difference here).

The meta-categories such as `(X\X)/X` are written in TheBench as `(@X\@X)/@X`. They are allowed with application only. Linguistically, only coordination and coordination-like structures seem to need them, which are syntactic islands with Ross (1967) escape hatches on syntactic identity, that is, things having the same case, therefore same category:

Meta category

(16) `and | x :: (@X\*@X)/*@X : \p\q\x. and (p x)(q x)`

This way we can avoid structural unification to do linguistic work; all of it has to be done by the invariant analytic structure. (Recall also that structural-reentrant unification is in fact semi-decidable. We simply use term unification.)

For the l-command, one important property is that its realization in analysis arises from the evaluation of the surface correspondents in the s-command and l-command, that is, surface structure and predicate-argument structure, therefore we cannot have a complex s-command (i.e. one with a slash) and simplex l-command (i.e. no lambda abstraction to match the complex s-command). For example, in the following, the '`\np`' has no l-command counterpart (some lambda), therefore unable to keep the correspondence going:

Category well formedness

(17) *`slept :: s\np : sleep someone`

However, the inverse, that is, having a complex l-command and simplex s-command, is fine, and common, for example

(18) `man :: n : \x.man x.`

This is because the analytics is driven by the syntactic category, that is, by the s-command. Not all lambdas have to be syntactic, but all syntactic argument-taking must have a lambda to keep the correspondence going in an analysis. (17) is not an analytically interpretable synthetic element. If the intention here were to capture the topic-dropped 'sleep,' for example *slept all day, what else could I do?*, it might be say `slept :: s : sleep topic`.

Limits of syntax semantics corr.

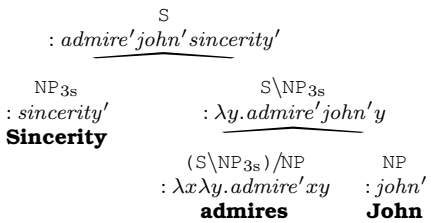
This constraint serves a much larger role in `MG` in trying to always give an account of meaningfulness as a consequence of grammaticality.

We have covered all three types of elements that can enter a monadic grammar so that analytics can compare the objects of grammar compositionally. Essentially, having to compose in a monad forces the conditions on the elements of the well-formedness conditions we have discussed. Object comparison through functions and relations only is the hallmark of category theory in mathematics, the monadic grammar being its subspecies. Comparison can serve both interpretation and production, also bottom-up and top-down parsing. (That is, 'comparison' cannot be reduced to parsing.)

Display of analyses & ranking

Analyses are displayed in a flattened format for reasons of screen space. For example, the `a-command` displays the analysis tree in (19) as (20a), assuming the grammar in (b) is loaded by the `g-command`, and used by the `a-command` and `,-command` (c). NB. the slash modalities in the textual notation in displays and grammars. Basic category's features are not displayed for reasons of space; they can be seen in the grammar (b). To get (a), we edit the grammar in (b) and do the steps in (c).

(19)



(20)

a.

```
Analysis 1
--------------
ELM   (sincerity) :: NP
        : SINCERITY
ELM   (admires) :: (S\NP)/^NP
        : (LAM X (LAM Y ((ADMIRE X) Y)))
ELM   (john) :: NP
```

```
        : JOHN
C1    (|admires|)(|john|) :: S\NP
        : ((LAM X (LAM Y ((ADMIRE X) Y))) JOHN)
C2    (|sincerity|)(|admires| |john|) :: S
        : (((LAM X (LAM Y ((ADMIRE X) Y))) JOHN) SINCERITY)

Final l-command, normal-order evaluated:

    ((ADMIRE JOHN) SINCERITY) =
    (ADMIRE JOHN SINCERITY)
```

b.

```
john      | n :: np[agr=3s]: john
sincerity | n :: np[agr=3s]: sincerity
admires   | v :: (s\np[agr=3s])/^np : \x\y.admire x y
```

c.

```
/\ g saj.gram
no errors in grammar text, proceeding with set up..
please check the /var/tmp/thebench/saj.gram.log file for information.
saj.gram.src file generated
grammar loaded from /var/tmp/thebench/saj.gram.src; ready for analysis
/\ a sincerity admires john

Number of analyses: 1
Done. Try , command for results
/\ , 1
```

In (a), `ELM` means 'element' (of grammar). If you see other material in the same location, it would indicate the rule used for comparison (some form of function composition, because that is all a monad does; see `MG`, Appendix A.). The names are listed in (10). They can also be the names of relational categories assigned by the grammarian. Lambdas are internally nativized for processing. The `,-command` shows all analyses if a specific one is not demanded. The number of analyses is not ordered or ranked in any way. Grammar entries are not ordered either. The order of relational categories in a grammar may be important; they are applied in sequence.

The `r-command` ranks the analyses and shows only the top-ranked (i.e. most likely) analysis after training. For example, in the following sequence of commands, the same expression above is ranked, assuming it comes from a trained grammar. Notice the display of weights to trace ranking.

(21)

```
/\ g saj.gram
no errors in grammar text, proceeding with set up..
please check the /var/tmp/thebench/saj.gram.log file for information.
file saj.gram.src exists in /var/tmp/thebench/, regenerating it.
saj.gram.src file generated
grammar loaded from /var/tmp/thebench/saj.gram.src; ready for analysis
/\ r sincerity admires john
Done. Try # command for results
/\ #

Most likely l-command for the input: (sincerity admires john)

  ((ADMIRE JOHN) SINCERITY) =
  (ADMIRE JOHN SINCERITY)
```

```
Cumulative weight:  8.0

Most probable analysis for it: (3 1 1)
------------------------------
ELM      1.0  (sincerity) :: NP
        : SINCERITY
ELM      1.0  (admires) :: (S\NP)/^NP
        : (LAM X (LAM Y ((ADMIRE X) Y)))
ELM      1.0  (john) := NP
        : JOHN
C1         2.0  (|admires|)(|john|) :: S\NP
        : ((LAM X (LAM Y ((ADMIRE X) Y))) JOHN)
C2         8.0  (|sincerity|)(|admires| |john|) :: S
        : (((LAM X (LAM Y ((ADMIRE X) Y))) JOHN) SINCERITY)

Final l-command, normal-order evaluated:

    ((ADMIRE JOHN) SINCERITY) =
    (ADMIRE JOHN SINCERITY)


Most weighted analysis : (3 1 1)
--------------------------
ELM      1.0  (sincerity) :: NP
        : SINCERITY
ELM      1.0  (admires) :: (S\NP)/^NP
        : (LAM X (LAM Y ((ADMIRE X) Y)))
ELM      1.0  (john) :: NP
        : JOHN
C1       2.0  (|admires|)(|john|) :: S\NP
        : ((LAM X (LAM Y ((ADMIRE X) Y))) JOHN)
C2         8.0  (|sincerity|)(|admires| |john|) :: S
        : (((LAM X (LAM Y ((ADMIRE X) Y))) JOHN) SINCERITY)

Final l-command, normal-order evaluated:

    ((ADMIRE JOHN) SINCERITY) =
    (ADMIRE JOHN SINCERITY)
```

The reason it always shows two analyses although the top-ranked solution is sought is that, the most likely meaning for the expression is a maximum of sum of products of weights (probabilities and counts), therefore, in principle, the analysis with the highest individual value may not be in the sum-maximum choice. If they are different analyses, it is worth looking at why. The cell notation is e.g. `(3 1 1)` meaning row 3 of the CKY (analysis) table, column 1, analysis 1. These are for debugging and model development purposes. To make sense of the weights for example in the most probable analysis above, notice that all `ELM` entries in the analysis have the parameter value 1.0, and the category-meaning pairs that is used to conpare the most probable analysis is used 8 times. (If all 8 uses had the parameter value say 0.5, the total weight would be $8 \times 0.5 = 4$.) In between, for example `admires john` makes use of two `ELM`s with parameter 1.0, adding up to 2.0. As the parameters change in training, these products therefore ranking would change. There is also a plugin to add more features than use-count as long as they can be discerned from an analysis in the CKY table; look up the definition and use of the function `plugin-count-more-substructure` in the file `bench.lisp`.

Why two analyses?

The appendices explain that **the real tug of war in training and ranking is be-**

What really competes in learning?

tween the categories of the same form. If a form has no competitor category, even the worst category for the form would survive in ranking.

All training is category-driven; there is no bypass for special attention to a particular form. This is one way to treat both **grammars** and their **models** as **transparently studiable**.

Grammars, models, analysis & ranking

7 What is a model of grammar?

A scientific model is often described as a simplified representation of reality. This description is philosophically loaded. It also does not explain why we build models. At least in natural sciences, a model prepares a theory or an idea for experiments. These can be hands-on experiments or thought experiments, but experiments nevertheless.

The point is that if a theory or idea is modelable in this sense, it would not be its believers and practitioners only who can test it out; anyone with sufficient interest and stamina must be able to attempt independent assessment of claims. I don't think the social semiotic that is assumed (by MG) to be behind categorial intuition—choice—is modelable yet in this sense,[8] but its consequences probably are. And since a grammar concrete or abstract is this consequence in MG, we can model it.

For that to work, we would need a nomenclature arising from the theory and a modeling language. For TheBench, the nomenclature has been described throughout this manual and in MG.

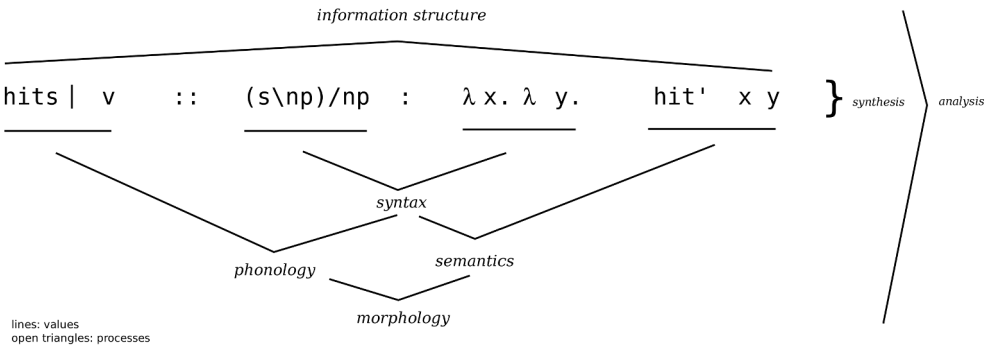
The modeling language is best disambiguated from the start so that believers and non-believers alike can formulate a model avoiding open interpretations about what each modeling dimension means. That way, the results would be reasonably interpretable by everyone. This means that, for TheBench to address these concerns, it would have to be specific about what the ingredients of any model, which are the elements of grammar, are, in each dimension. These dimensions can be shown entries like the following, repeated from (12).

(22) `hits | v :: (s\np)/np : \x\y.\so{hit} x y`

They are:

Phonology, morphology, syntax, semantics, and information structure in a model

(23)



(24) a. Phonology (the process connecting the material before '|' with the syntactic process—see (23)). The conception of phonology here is the study of the medium and its structure for transmission and intake of command relations. Signs of sign languages are naturally covered, gestures are not; we don't know whether their reference needs a command relation.

Phonology brings to discussion the notion of adjacency, whether it is material or abstract. Adjacency of comparison in MG is not a commitment to segments, suprasegments or any other phonological block. It just means that all elements of grammar must be independently specifiable and comparable, that is, connectable. And they are accesible only through their categories. For example,

[8] See Bozşahin (2024) for a personal attempt to make it modelable.

if there is a way to make generative grammar's big PRO or small pro as elements (say as bundle of phonological features) that are independently comparable dual command relations, and we do so without overgeneration, that would be a monadic capture of these ideas. Adjacency is an abstract notion that delimits analyses.

b. Morphology (the process connecting phonology and semantics). The '::' is the monadic symbol for *type construction*. Morphology is the only type constructor, with '::' structuring the material that precedes with the material that follows and specifying a label (here, 'v'), a part of speech if you like. The conception of morphology here is the constructor of reference and dereference, whether or not the language has 'bound operators.' Reference requires two command relations specified if it is structured reference, for any element.
   In relation to traditional branches of morphology, one may think of derivational morphology as redirecting reference and dereference on other material, inflectional morphology as narrowing reference of other material or further specifying dereference, and word formation as rebuilding and restructuring reference. Therefore even a language with barest of forms, i.e. no inflections, no derivations, no word formation, has morphology because the barest form too would have to have two command relations specified to construct reference. Constructing dereference (not 'deconstructing reference') is the interpretive process of that, reinstating meaningfulness.

c. Syntax (the process connecting the syntactic type with the lambdas). The conception of syntax here is the study of transmission and reinstatement of reference through command relations. It is therefore necessarily structured, because of having to carry the command relations. The syntactic command is represented in the syntactic type. Its correspondence is in the lambdas, the roles of which are up to semantics. Syntax does not build reference. Heads of syntactic constructions build reference to structure and reference to content of course, for example the relative marker, but that is morphology as understood here.

d. Semantics (the process connecting the syntactic process to predicate-argument structure). It is conceived as the study of the source of reference and structured representation of reference to who does what to whom. 'Content' enters semantics through reference. (It is probably easier to talk about form and reference rather than form and content in syntax-semantics relations because of this. What we write in a predicate-argument structure is not really content but reference to content.)
   For MG in particular, although we may be tempted to keep the lambda bindings out of semantics, because they prescribe the connection to syntax rather than determine semantic structure, not all lambda bindings are necessarily syntactic (see e.g. (18)), therefore it is best to consider all of them as part of semantics. It will be obvious from a particular correspondence how many of the outermost lambdas are syntactic. Therefore lambda binding as glue-language is always disambiguated in an element.

e. Information structure (the process connecting the whole material). It is the study of how the form and reference are packaged into a message and how such messages are unpacked presuming they were structured. It may involve phonology (e.g. pitch accents, boundary tones), morphology (e.g. topic or focus particles), syntax (e.g. reordering) and semantics (e.g. theme-rheme marking).

Grammar's processes and representations

The descriptions above may give the impression that these levels are levels of representation. They are actually processes. The processes related to grammars consisting exclusively of entries are as follows. Grammar's representations are underlined in (23). They are synthetic in the sense that they are already put together in the form of a correspondence.

Therefore, in a model of monadic grammar, syntax does not construct reference. It transmits its correspondence. Morphology does not transmit reference. It connects two processes, one that transmits reference (connecting syntax and phonology) and the one

that builds it (semantics). There is no competition between syntax and morphology. In fact there is no competition between any process, or stepping on each other's toes.[9]

Information structure is structured messaging because it would have to involve command relations, the extent of which is not fixed a priori. Altogether analysis is the process of revealing the breakdown of the wholes represented in the synthetic elements. Its only invariant is function composition, which applies to all the processes mentioned. I hope these aspects further clarify the modeling landscape of `TheBench`.

A grammar model must be unambigously interpretable by modelers of all persuasion, all the way down to its elements of grammar and up to the processes, if we want to make the theory behind the model testable by scientific means. I hope this is not only true for `MG`.

8 Exploration: Case, agreement, grammatical relations, grammar modelling

There are some aids in `TheBench` to explore monadic analytic structures.

1. We can explore what kind of syntactic case and similar structural functions arise given a grammar, from its verbs and verb-like elements. These functions can be conceived as asymmetric relational categories, showing the understanding that if we have one category for an argument we also have other categories for the argument, which is a sign of mastery of argument-taking in expressions.

The '`c command`' of `TheBench` generates these functions from a grammar (loaded by the '`g command`'), taking a list of parts of speech as input from which to generate the second-order functions. (Presumably, these are the parts of speech of the verbs and verb-like elements.) These extra functions are saved in a separate textual grammar file with extension '`.sc.arules`'. The name designates that all of them are asymmetric relational categories. `Case`

These categories can be merged manually with the grammar from which they are derived. The reason for not automating the merge is because you may want to play around with them before incorporating them into an analysis. It is a textual file, therefore editable just like any grammar text file. Many surface structures follow from incorporating them into a grammar, which you can check with the '`a command`' (for 'analyze').

After the merge, analyses will reveal many more surface configurations as a consequence of synthetic case in the language under investigation. This way of looking at case renders generalizations about case, for example lexical, inherent and structural case (Woolford, 2006), as bottom-up typological universals arising from a range of class restrictions on the verb, from most specific to least. `MG` has more to say about these aspects.

2. We can preview the syntactic skeleton of a grammar in `TheBench`. '`k command`' goes over all the elements of the currently loaded grammar, and reports the kind and number of distinct syntactic categories in it. `Grammar skeleton`

For brevity it does not report features of a basic category. This much is certain though: if two categories reported by this command look the same but reported separately, it would mean that they are distinct in their features, to the extent that they would not match in analysis either. For example, `s\np[agr=3s]` and `s\np[agr=1s]` are distinct, although their featureless version looks common: `s\np`.

One aspect of the '`k command`' can be used to find out the patterns in the grammar's categories, and to find out why some categories are distinct eventhough they might look identical as reported by the '`k command`': for every distinct syntactic category reported,

[9] It seems that all this talk about 'competition' and 'resolution/rebracketing' is an ideological commitment.

it enumerates the list of elements that bears it. These forms are presumably indicators of common syntactic, phonological and/or morphological properties, for example agreement classes and bound versus free elements.

TheBench is a tool for a very bottom-up view of grammar. Naturally, more than N, V, A, P distributions will be reported. (In fact, TheBench has no built-in universal categories or features. If you don't use these four categories at all, it is fine.) To see the basic category inventory, in addition to the inverted list display of the 'k command', take a look at the output of '! command', which reports the basic categories in the grammar and the list of all features in them, and attested values.

No built-in category

3. There is no morphophonological analysis built in to TheBench. The morphological boundedness of a synthetic form can be designated with '+' in a and r commands for convenience. It goes as far allowing in analysis/ranking entering the surface tokens such as MWEs either as 'dismiss ed' or 'dismiss+ed', if you have 'ed' as an entry in the grammar. ('+' is a special operator for this, i.e. the entry is not assumed to be '+ed'.) Keep also in mind that TheBench has no notion of 'morpheme'—you must model that if you want—but it has a notion of morphological structure: the category building on phonology to construct reference, Montague-style.

Multiword expressions (MWES)

The current '+' notation is just for some convenience until somebody comes up with a categorial theory of morphophonology for example along the lines of Schmerling (1981); Hoeksema and Janda (1988). Till then, we won't get 'insured' from 'insure+ed', or Turkish harmony distinction'avlar' (game-PLU) and 'evler' (house-PLU).

The idea behind this self-distributed morphology is that languages of the world can distribute themselves across a grammar as they see fit by employing a mixture of functional and relational categories, rather than some pre-determined cascade of morphological and syntactic operations (e.g. lexical insertion). Morphology is not conceived as a convenient tokenizer; it is the category constructor.

Self distributing morphology

4. We can turn the textual form of TheBench grammar into a set of data points each associated with a parameter. These parameters are parameters in the modelling sense, characterizing a data point. To do this, TheBench turns a textual grammar into a 'source form', which is a COMMON LISP data structure, basically Lisp source code. Such files are automatically generated and carry the file extension of '.src'. This is an inspectable file, but it is much easier to inspect when it is turned into a textual grammar in TheBench notation, which is what the 'z command' does.[10] The resulting textual file will be much like the original textual grammar, without comments, and with keys and initial value of the parameter added on the right edge of every entry, in the form of for example '<314, 1.0>' where 314 is the key and 1.0 is the parameter value. The keys are unique to each entry, therefore having multiple categories for the same form is possible; they would be distinct entries even if they are identical in text.

Entry keys & parameters

These values are not probabilities; they are weights. The ranker (the 'r command') turns them into probabilities to arrive at most likely analyses, given the weights. The trainer adjusts these weights depending on the self-supervision data. Therefore it is important to know that entries *with the same phonological form* compete with their weights in making themselves enter an analysis. (In other words, there is no built-in determinism, mapping each phonological form to only one interpretation.)

Weights & probabilities

The system makes the initial assignment automatically to ensure the uniqueness of the keys and completeness of parameter assignment.[11] For example, when you inspect

[10] The 'z command' can also transform legacy '.ded', '.ind' and '.ccg.lisp' grammars of CCGlab to TheBench grammar format. Please change the top line of such grammars to '(defparameter *current-grammar*' and put the result in /var/tmp/thebench directory before running the command. The result will be put back in your working directory as a text file in the monadic format.

[11] If you use a specific key for a particular entry in your grammar text, say for easier tracking of a particular item during model training, the system respects your key choices and parameter initialization. I recommend not using this feature extensively to avoid key clashes. Uniqueness of personally assigned keys is not

User specified keys

the regenerated textual grammar containing (12), you may see something like:

(25)
```
likes  | v :: (s\^np[agr=3s])/^np : \x\y.like x y <120, 1.0>
#np-raise np[agr=?x] : lf  --> s/(s\np[agr=?x]) : \lf\p. p lf  <34, 1.0>
runs | tense :: s[t=pres,agr=3s]\np:\x.pres run x  <2, 1.0>
ran | tense :: s[t=past]\np:\x.past run x <76, 1.0>
```

This file is just as editable and processable as the original textual file containing the grammar. However, notice the difference from (12). The symmetric relational categories are compiled into separate entries in (25), and their link is preserved by using the name of the relation as 'part of speech' in both entries, in this case `tense`.

The point of creating different data points is that, when trained, these elements will participate in different analyses therefore need parameter values of their own. Using the preserved link (same tag) we can explore reconstruction of the paradigm intended by '`<-->`'. The possibility of using the same relation name to capture a paradigm facilitates this exploration.

There is one more change in the reconstructed text file: every entry's capitalization is normalized to lower case, except the phonological material, whose "orthographic case" is preserved. This is one lame attempt to indicate that COMMON LISP's default capitalization of values without us asking for it would not make any difference to an analysis.[12]

5. Taking the textual source of a developing grammatical analysis and turning to modelling with it is the idea behind the textual reconversion of a grammar source. The reconstructed source has the symmetric relations turned into separate entries with same 'part of speech': the relation's name. They will have their own parameter values. This is probably the last step after grammar development, when it is time to move on to grammar training to adjust belief in elements, using parameters. This is why the command that does this is called the '`z command`'.

`Fine-tuning & modeling`

Training a grammar with data updates the parameters, which can then be used for ranking the analysis, that is, for choosing from the possible analyses the most likely one after training. The method used is the sequence learning of Zettlemoyer and Collins (2005). It is basically a gradient ascent method. It places the whole trained grammar in probabilistic sample space of possible grammars.[13] The appendices explain the meaning of parameters increasing, decreasing or remaining unchanged.

The self-supervision pairs are correspondences of phonological forms and their presumed correct predicate-argument structures, sometimes called in computational work the 'gold annotation'. However, the term would be misleading, because there is no annotation or labeled data.

The term 'supervision' needs clarification in the context of `TheBench`. It means that we *presume* that that data is known to hold some correspondence of expressions and predicate-argument structures, much like Brown (1973), Brown (1998) and Tomasello (1992) did when they started analyzing child data—it is indeed an adult assumption coming from psycholinguistic analysis. The predicate-argument structure is a specification of the presumed references of the phonological item, written in a text file one line per entry, for example:

(26)
```
Mary persuaded Harry to study : persuade (study harry) harry mary
Mary promised Harry to study : promise (study mary) harry mary
Mary expected Harry to study : expect (study harry) mary
```

checked. If you do use the special key assignment feature, make sure that there is no keyed element after any unkeyed element, that is, pile unkeyed elements at the end of the grammar file. This way your assigned keys will be seen before new key assignment begins.

[12] I could make it case-sensitive, to preserve both the analyst's and Lisp's cases, but this would be quite error prone: Is `AdvP` same as `advp`?

[13] Linguistics alert: These are not probabilistic grammars, things in which a category would be uncertain. They are grammars with clear-cut categories which are situated by model selection in the probabilistic space of possible grammars. The idea is not too far from the continuity hypothesis in language acquisition by which a child moves from one possible grammar to another, apparently nondeterministically; see Crain and Thornton (1998).

Here, the material before the colon is the proxy phonological form (therefore "orthographic case"-sensitive). Multi-word expressions (MWEs) must be enclosed within "|' if they are considered to be referentially atomic, e.g. `|kicked the bucket|`. (After all, this is supervision in the sense above, so we assume its reference is fixed, we just need to find out which part of the predicate-argument structure corresponds to it, given a grammar with that MWE.) MWEs

The material after the colon is the presumed predicate-argument structure of the whole expression. It has the same format as in grammar specification. The '`t command`' (for 'train') updates the parameters of a grammar-turned-into-a-model by this method. We can pick from the candidate models by looking at its output (known as model selection), which is a collection of plain text grammars (the number of candidates is specified by you), with parameters for every entry. We can then load the chosen grammar, and rank analyses with it using the '`r command`'.

The input to training is as many as number of entries in a file with lines like those in (26). We suggest minimal (required) parenthesization to avoid near misses in training; `TheBench` will internally binarize the predicate-argument structure anyway. Exact repetition of an entire line constitutes a separate self-supervision entry. This is useful—in fact required—in analyzing child-directed speech, where repetition is very common.[14] Model training, selection, and deployment

6. The tools used for training a grammar on data such as (26) are triggered from `TheBench` command line, but they work offline, using the same processor code. These training sessions can last quite long, sometimes hours, sometimes days or weeks depending on the grammar size, the number of experiments attempted and the amount of supervision data. (That's because of the computer speed.) We don't have to stay in the originating session to keep the experimental runs going. They operate independently, using the `nohup` facility.[15] During these runs, all information must be available offline. To do this, we prepare an 'experiment file'. It has a strict format, for example: Experiment files

(27)
```
7000 4000 xp 1.2 1.0 nfp nfparse-off
4000 2000 10 0.5 1.0 bon beam-on
4000 2000 10 0.5 1.0 boff
```

This specification will fetch three processors if it can because three lines of experiments are requested, one using 7GB of RAM in which 4GB is heap (dynamic space needed for internal structures such as hash tables, etc.), the others with 4GB RAM and 2GB of heap. As a guideline, a larger heap helps to run analyses of longer expressions, which usually arises when case functions are added to a grammar.

These parameters are useful to run the experiments on machines with different powers and architecture (number of cores, amount of memory, size of heap) without changing the basic setup.

The '`t command`' specifies which grammar and supervision will take place in the experiments. (Therefore every experiment in one file runs on the same grammar-training data.) The supervision file must be in the format of (26). The rest are training parameters: `xp` means use of the extrapolator, which uses a predetermined number of iterations for the gradient ascent in implementing the Cabay and Jackson (1976) method.

The second experiment does not extrapolate; it iterates 10 times. Every iteration updates the gradient incrementally. Then comes the learning rate (1.2 in the first experiment), which is the distance the gradient travels in one step ('the jump distance'), the learning rate rate (1.0 in the first experiment), which is how later iterations affect the jump distance, the prefix of the log file e.g. `nfp` (the actual name of the log file is

[14] Assigning different semantics and reference to every repetition of an identical expression is the holy grail of child language research. We have no theories for that as far as I know.

[15] `nohup` is for 'no hang up', which is one more reason why a `linux` system or subsystem is indispensable for tools such as this. One caution: Do *not* exit your session with control-D, which would kill the subprocesses in the linuxsphere. Just let it die or hang, or close the window normally, e.g. by clicking the red button in the red-yellow-green window menu.

prefixed with that, adding training parameters as suffixes of the name for easier identification of experiments that will be saved in the end), and the function to call before training starts, which is optional, as in the third experiment. In the first experiment it is `nfparse-off`, which turns normal form parsing off, which is a feature in the processor, implementing the algorithm of Eisner (1996) for reducing compositions after they come out of the monad. This affects the number of analyses generated and reported.

The full list of such functions is given at the back, which are callable from the command interface as well the '`l command`'. (For processor functions with more than one arguments, calling is a bit more complex; see the `cl4py` Python library documentation.) In the second experiment, the function to call prior to experiments is `beam-on`. This function focuses the gradient on items with largest weight changes during iterations, avoiding the consideration of elements that change very little (no change is effectively handled by default). The results may be less precise because of this but it reduces the search space for the gradient. It is sometimes a lifesaver in eliminating excessive runtimes or out-of-memory errors.

7. Studying referentiality of all kinds along with configurationality in one package seems to be one way to avoid the purportedly dichotomic world of verbs-first and nouns-first approaches in language acquisition. `TheBench` is designed primarily to rise above these assumptions with the help of modelling.

By combining explorations **4–5–6** it is possible to have another look at event-and-participant attempts to explain language acquisition, for example those of Brown (1973); Tomasello (1992); Brown (1998); MacWhinney (2000); Abend et al. (2017), to compare with participant-centric approaches such as Gentner (1982).

In such an undertaking, the observables would be the phonological form on the left, much like in (26), and the analyst's conclusion about what they would mean would be the predicate-argument structure on the right. There would be no labeled or hidden variables such as syntactic labels or dependency types. The mental grammar which is presumed to arise in the mind of the child would be proxied as a grammar much like in the earlier sections, which would be trained on the data such as above for model selection.

`Syntax as latent variable`

9 Recommended work cycles

Three work cycles are reported here from personal experience, for whatever their worth.

The first one is for the development of an analysis by checking its aspects with respect to the theoretical assumptions. This would be the simple cycle in (28a), with '`k`' and '`!`' commands sprinkled in between (not shown here). It would edit and load a grammar (`g`), analyze an expression with it (`a`), and display the results (`,`).

`Writing, editing, exploring grammars`

(28) a. `edit - g command - a command - , command`
b. `edit - g command - c command - edit/merge - a command - , command`
c. `edit - t command - z command - g command - r command - # command`

The second cycle (b) would be for studying a grammatical analysis in its full implications for surface structure when the class of verbs is large enough. 'Merge' here presumably merges case functions with the grammar. When we generate case functions using '`c command`', the currently loaded grammar have access to them so we can do analyses with them in the current session. However, they are not added to the grammar's text file, so the grammar is left intact when the session is over. The merge is left up to you. The case functions are saved in a file to facilitate that.

The third one, (c), is for training a grammar to see how it affects the ranking of analyses. The '`z command`' here presumes that one trained grammar is selected for ranking, so the outputs of the '`t command`' have presumably gone through some kind of model selection after training, that is, choosing one of the grammars with updated weights, either manually or by a method of model selection.

The 't command' in (c) will fork as many processes as the number of experiments requested; NB. discussion around (27). It's best to use it offline. For that, put it in the commands files and use the '@ command' for batch processing. Waiting online can be painful depending on the number of experiments and data sizes. Just don't do control-D on TheBench interface if you use it online; it will kill the subprocesses. Let the terminal hang. Don't close the lid on a laptop; that would suspend all work. (Speaking of laptops, which are more and more designed toward efficient handling of video, audio and graphics using specific processors for them, it is best to use a more general high-speed multicore/multiprocessor system to do the training. Many experiments that ground a laptop to a halt worked effortlessly when I ran them on a powerful desktop.)

TheBench command macros

Laptops; no control-D

If you are training a grammar on a large supervision dataset, and want to find out how the top-ranked analyses fare with the gold pairings of expressions and meanings, one option of '# command' combined with the '> command' can be helpful. Before ranking, turn on logging with the '> command'. Then for '# command' in a batch command file use the 'bare' option to eliminate verbosity. Minimal amount of extra text will be saved in the *processor's log file*, which you can easily eliminate. Don't forget to turn off logging at the end, using the '< command'.

Logging

One advice about the '/ command'. This command clears the /var/tmp/thebench directory where the internal results of TheBench are kept. Do not use this command if you started an experiment, either online from the interface or in batch mode (see glossary). These commands create files that are needed later on; see the end of §4 for file organization in TheBench.

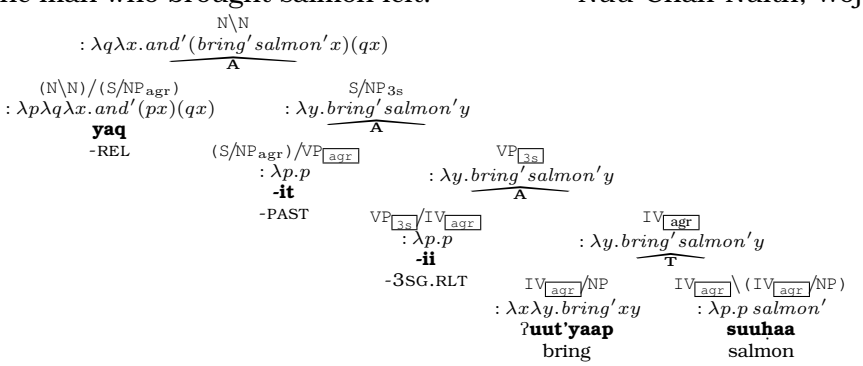
To clean or not to clean

10 To do

I have a personal to-do list for collaborative work. I mention here the technical ones which require programming. For typology work, just drop me an email please.

1. Implementing the 'dashed boxes' of TheBench. 'Analysis' as understood in TheBench is a wholistic process, where every part-process is interpreted in the light of the whole in check all the time, at every step. Analyses can be displayed in the following full form.

(29) yaac-waas-it-ʔiš čakup-ʔi yaq-it-ii ʔuut'yaap suuḥaa
walk-outside-PAST-3SG man-DET -REL-PAST-3SG.RLT bring salmon
'The man who brought salmon left.' Nuu-Chah-Nulth; Wojdak 2000:274

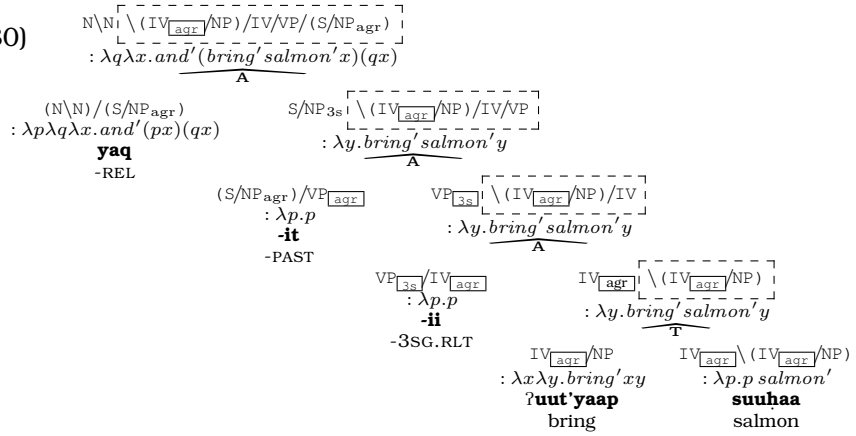


However, the original conception of categorial grammar was not just a better and simpler alternative to phrase-structure grammar, with or without movement. (I suppose this rather unfortunate overinterpretation emanated from Bar-Hillel, Gaifman and Shamir 1960/1964.) Every step of analysis is supposed to keep the whole in check, with parts showing *subprocesses* themselves. This was the connection to Gestalt Psychology, see for example Wertheimer (1924).

So what we really want to implement in TheBench is showing how the whole can be kept in check at every step. The dashed boxes in the following analysis of the same example show how it can be done. It has not been implemented in TheBench yet.

The Gestalt whole in check

(30)



The overall result on the top in its full form does not mean that for example (S/NP_{agr}) is followed in sequence by VP, then IV, and so on. It shows the structural unfolding of the entire expression, that is, the part-processes. The end result is on the left, unboxed, undashed.

Such structures would be necessary but not sufficient to situate reference in context.

2. Production/generation. Categorial production can be seen as expressing an intended complex reference by choosing elements from grammar that would serve that reference. It would be more than expression of thought. It is more like environment control by language.

One way to start studying such processes is to see how far compositionality in grammar can be put to work on this task. To do this, it would be nice to have a "structural chunker," one that would take the triplet <word, category, size> and produce an expression, where word is the chosen phonological left edge of the chunk, category is the syntactic category of the overall result intended, and size is the upper limit on the chunk in terms of number of elements in it. The upper limit would be needed to make the problem decidable because the category can be endotypic, for example S\S, therefore the process is potentially recursive.

Generation is usually understood that way in language technology with phrase-structure grammars, because they tend to take care of distributional adequacy first and worry about reference later. In categorial grammar however, it is half the story, because reference itself would reveal the item choice in the first place.

3. Encapsulating the monad. A true monad is a hermetic seal. We cannot peek inside to see the intermediate results. In the current state of TheBench, we can. That's because it is easier to debug.

However, turning the current state of the implementation to a true monad requires more than employing a monad library in Python or Common Lisp. The categories that are input to the monad would have to be lifted in syntax and semantics.

4. IPA support. I wish I had the resources to do that. For the moment, you can make use of UTF8 tools to write IPA. If anyone wants to develop direct IPA support for TheBench, please send me a pull request.

Acknowledgments

I thank Python, Lisp, StackOverflow and StackExchange communities for answering my questions before I ask them. The l-command evaluator in the processor is based on Alessandro Cimatti's abstract data structure implementation of the lambda calculus in Lisp, which allows us to see the internal make-up of the l-command. David Beazley's

`sly` python library made `TheBench` interface description a breeze. Marco Heisig's `cl4py` python library made Python-Lisp communication easy, which allowed me to recycle some Lisp code. Luke Zettlemoyer explained to me his sequence learner in detail so that I can implement it on my own. Discussions with Chris Stone clarified some of the tasks in to-do list. Alexander Fedorov of the SBCL team helped me get one `TheBench` mystery straightened. Deniz Akdemir helped me fix the problems I have been having with the installer. Thanks to all colleagues. I blame good weather, hapless geography and the cats in the neighbourhood for the remaining errors.

Appendix A: Mathematics of the `r-command`

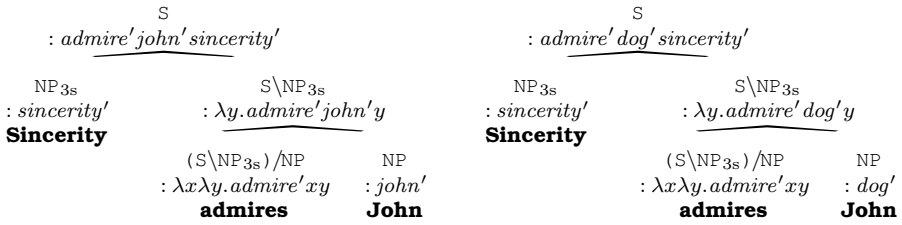
The `r-command` ranks the analyses depending on weights of the grammar elements (synthetic elements with function categories and paradigmatic elements with relational categories). It takes a surface expression and delivers the most likely l-command for it if it is grammatical. That is, it is ranking combined with analysis.

Once parameters are reestimated and a grammar model is chosen, we can assess the quality of the chosen model using a parse-ranking algorithm. The one we use is summarized from Zettlemoyer and Collins (2005):

(31) $\arg\max_L P(L \mid S;\bar{\theta}) = \arg\max_L \sum_A P(L, A \mid S;\bar{\theta})$

where S is the expression to be ranked, L is the l-command for it, A is a sequence of analyses for the (L, S) pair, and $\bar{\theta}$ is the n-dimensional parameter vector for a grammar of size n (the total number of elements).

Example: Suppose we have two alternative analyses (As) for the same expression:



There are two Ls (note that John entries differ). Each has one A. It is a simple choice for the analysis that maximizes the l-command probability of the expression:

$\arg\max_L P(L \mid$ sincerity admires John$;\bar{\theta}) =$

$Max_L P($: $admire′\, john′\, sincerity′$,
S\NP3s : $\lambda y.admire′\, john′\, y$ **admires john**, S : $admire′\, john′\, sincerity′$ **sincerity admires john**
· · · · · · NP : $john′$ **john**
| sincerity admires john$;\bar{\theta})$

$P($: $admire′\, dog′\, sincerity′$,
S\NP3s : $\lambda y.admire′\, dog′\, y$ **admires john**, S : $admire′\, dog′\, sincerity′$ **sincerity admires john**
· · · · · · NP : $dog′$ **john**
| sincerity admires john$;\bar{\theta})$

How do we measure $P(L, A \mid$ sincerity admires john$;\bar{\theta})$? It is induced from the following relation of probabilities, frequencies and parameters.

(32) $$P(L, A \mid S;\bar{\theta}) = \frac{e^{\bar{f}(L,A,S)\cdot\bar{\theta}}}{\sum_L \sum_A e^{\bar{f}(L,A,S)\cdot\bar{\theta}}}$$

where $\bar{f}$ is a vector of 3-argument functions <$f_1(L, A, S), \cdots f_n(L, A, S)$>. The functions of $\bar{f}$ count local substructure in A. By default, f_i is the number of times i (an element or relation) is used in A for analyzing S leading to L, sometimes called the *feature* i.

We can then use model assessment methods such as cross-validation, precision, recall and f-measure. For assessing language acquisition, one generalized testing method is particularly relevant: how many of the ranked parses deliver the analysis assumed in the child's supervision data, as the top-ranked analysis? If there are many different uses of the same word, which is typical in child-directed speech, it would be much less than 50% that the correct correspondence can be captured without grammar training.

Appendix B: Mathematics of the `t-command`

Data parameters (weights of grammar elements) can be reestimated from training data of (L_i, S_i) pairs where L_i is the meaning (l-command) associated with sentence S_i. This is what the `t-command` does.

It takes form-meaning pairs (and nothing else) as training data, for example in `TheBench` format:

```
Mary persuaded Harry to study : persuade (study harry) harry mary
Mary promised Harry to study : promise (study mary) harry mary
Mary expected Harry to study : expect (study harry) mary
```

Notice that there are no syntactic or morphological annotation. The *log-likelihood* of the training data is:

(33) $O(\bar{\theta}) = \sum_{i=1}^{n} log\, P(L_i \mid S_i; \bar{\theta}) = \sum_{i=1}^{n} log(\sum_A P(L_i, A \mid S_i; \bar{\theta}))$

To see how likely our training data is according to our grammar, we analyze S_i pair by pair and add up all analyses (A) that led to L_i.

We can see how syntax is marginalized by summing over all derivations A of (L_i, S_i). For individual parameters we look at the partial derivative of (33) with respect to parameter θ_j. The local gradient of θ_j with feature f_j for the training pair (L_i, S_i) is the difference of two expected values:

(34) $$\frac{\partial O_i}{\partial \theta_j} = E_{f_j(L_i, A, S_i)} - E_{f_j(L, A, S_i)}$$

The gradient will be negative if feature f_j contributes more to any analysis than it does to the correct analyses of (L_i, S_i). It will be zero if all analyses are correct, and positive otherwise. Ascending values are what we seek.

The expected values of f_j are therefore calculated under the distributions $P(A \mid S_i, L_i; \bar{\theta})$ and $P(L, A \mid S_i; \bar{\theta})$. For the overall training set, using sums, the partial derivative is:

(35) $\frac{\partial O}{\partial \theta_j} = \sum_{i=1}^{n} \sum_A f_j(L_i, A, S_i) P(A \mid S_i, L_i; \bar{\theta}) - \sum_{i=1}^{n} \sum_L \sum_A f_j(L, A, S_i) P(L, A \mid S_i; \bar{\theta})$

Think of this gradient search as a way to investigate the *Continuity Hypothesis* of Crain and Thornton (1998) in linguistics. Every model of grammar would be a possible grammar if the model follows from a theory of natural language grammar.

Once we have the derivative, we use *Stochastic Gradient Ascent* to reestimate the parameters:

(36) Initialize $\bar{\theta}$ to some value.
for $k = 0 \cdots N-1$
for $i = 1 \cdots n$
$$\bar{\theta} = \bar{\theta} + \frac{\alpha_0}{1 + c(i + kn)} \frac{\partial log\, P(L_i | S_i; \bar{\theta})}{\partial \bar{\theta}}$$

where N is the number of passes over the training set, n is the training set size, α_0 and c are the learning-rate parameters (respectively, learning rate and learning rate rate). In `TheBench` these are specified in *experiment files*; see §8.5, §8.6. This is gradient *ascent*, so initialize $\bar{\theta}$ accordingly. Default is 1.0.

Stochastic gradient search? Are our grammars stochastic? No. Every grammar is a proxy for *categorial* understanding of the form-meaning relation. *Linguistic grammars* are symbolic empirical species. Formal grammars are, ehm, formal species. What is stochastic is the *space* of all (and hopefully only) human grammars.

Appendix C: TheBench commands in ?-command order

The ?-command's display is in Table 1.

The commands are performed on the following sample grammar, which is available in TheBench repository under examples/training.demo.basic.

filename: en.txt

```
% a simple grammar to show training workflow


% verbs
% good and bad cats, and polysemy for knows.
%   see the comments in the training output for these.

knows | v :: (s\np)/np : \x\y.know x y
knows | v :: (s\np)/s : \s\x.know s x
knows | v :: (s\np)/np : \x\y.know y x    % bad
knows | v :: (s\np)/s : \s\x.know x s    % bad

loves | v :: (s\np)/np : \x\y.love x y
loves | v :: (s\np)/np : \x\y.love y x    % bad

% subjects  and objects -- no type-raising

John | n :: np : john
Mary | n :: np : mary

% Quinean confusion: syn type is OK but LF is not

John | n :: np : dog
Mary | n :: np : cat
```

a-command

```
/\ a John knows Mary

Number of analyses: 8
Done. Try , command for results
```

c-command

```
/\ c v

Summary of compiling synthetic case functions for current grammar
There were NO errors/warnings in deriving second order functions
Number of elements in the grammar               : 10
Number of elementary functions considered       : 6
POS tags considered for synthetic case          : (V)
Number of second-order case functions generated : 6
Number of paradigmatic functions out of them    : 2

Case functions compiled OK.

Loaded grammar augmented with arules for case
 (NOT saved in its text file).


File: en.txt.sc.arules created; contains synthetic case rules for en.txt in the source format,
   for merging with grammar source text or for inspection.
```

e-command

```
/\ e print("hello world")
hello world
```

g-command

```
/\ g en.txt
no errors in grammar text, proceeding with set up..
please check the /var/tmp/thebench/en.txt.log file for information.
file en.txt.src exists in /var/tmp/thebench/, regenerating it.
en.txt.src file generated
grammar loaded from /var/tmp/thebench/en.txt.src; ready for analysis
```

i-command

```
/\ i lets-have-a-closer-look
grammar is pretty-printed to /var/tmp/thebench/lets-have-a-closer-look in internal representation
```

k-command

```
/\ k
The categorial skeleton of the current grammar


category  : NP
occurrence: 4  elements
elements  : (John / Mary / John / Mary)

--------

category  : (S\NP)/NP
occurrence: 4  elements
elements  : (knows / knows / loves / loves)

--------

category  : (S\NP)/S
occurrence: 2  elements
elements  : (knows / knows)

--------

Checksum: 10 (total of sums in category classes)
Total   : 3 distinct categories
Out of  : 10 entries (not counting asymmetric relational rules)

use > command beforehand to save the result
```

l-command

```
/\ l status
  ---------------------------
  Currently loaded grammar  : /var/tmp/thebench/en.txt.src
  current-grammar           : 10 items
  ARULES-TABLE              : 0 items
  CKY-HASHTABLE             : 20 items
  CKY-INPUT  for the table  : (John knows Mary)
  Most likely l-command     : NIL
  Its most likely analysis  : NIL
  Sum of weighted counts    : 0.0
  Most likely l-command's cell  : NIL
  Number of differing l-commands: 0
  Most weighted analysis        : NIL
  ---------------------------
```

Here, `status` is a Lisp function that takes no arguments, defined in `src/bench.lisp`.

o-command

```
/\ o date
Tue May 20 10:23:30 AM +03 2025
```

r-command

```
/\ r John knows Mary
Done. Try # command for results
```

t-command

```
/\ t en.txt en.spairs en.exp
no errors in grammar text, proceeding with set up..
please check the /var/tmp/thebench/en.txt.log file for information.
no errors in supervision data, proceeding with set up..
please check the /var/tmp/thebench/en.spairs.log file for information.

Training starts.
Please hit RETURN if the prompt is not back on.
You don't have to wait for the finish.
  You can continue to do other things in the interface
    or leave the session. Training runs in the background.

You can use 'top' command of linux to follow the progress
  Look for COMMAND column for processes named 'sbcl'.
  These are the currently running training processes.

Summary of experiments for locating results when done
------ Experiment 1 ------
result goes to: /var/tmp/thebench/nfp.xp.1.2a.1.0c.src
     log file : /var/tmp/thebench/nfp.log (contains summary of parameter change)
```

```
initial call  : nfparse-off (before the experiment starts)
------ Experiment 2 ------
result goes to: /var/tmp/thebench/bon.10.0.5a.1.1c.src
     log file : /var/tmp/thebench/bon.log (contains summary of parameter change)
initial call  : beam-on (before the experiment starts)
------ Experiment 3 ------
result goes to: /var/tmp/thebench/boff.8.0.5a.1.2c.src
     log file : /var/tmp/thebench/boff.log (contains summary of parameter change)
initial call  : noop (before the experiment starts)
--------------------------
If everything runs OK, you can re-generate source grammars from .src files
   use the z command for that

!!! Please do NOT hit ctrl-D in THIS terminal app. It would terminate the experiments.

/\ nohup: redirecting stderr to stdout
```

z-command

```
/\ z nfp.xp.1.2a.1.0c.src
grammar in /var/tmp/thebench/nfp.xp.1.2a.1.0c.src loaded

File: nfp.xp.1.2a.1.0c.src.462501.txt created; contains text for nfp.xp.1.2a.1.0c.src,
   minus comments and empty lines, with srules converted to entries.
grammar text is located in your working directory
```

@-command

```
/\ @ en.tbc batch-commands-in-en.tbc
Done. Check out the log in batch-commands-in-en.tbc.log
nohup: redirecting stderr to stdout
```

These are the contents of the file `en.tbc`, which are commands to be done in offlined mode.

```
pass train en.txt on en.spairs using experiment parameters in en.exp
t en.txt en.spairs en.exp
l show-config
```

,-command

```
/\ , 6


Analysis 6
--------------
ELM   (John) :: NP
        : JOHN
ELM   (knows) :: (S\NP)/NP
        : (LAM X (LAM Y ((KNOW Y) X)))
ELM   (Mary) :: NP
        : MARY
C1    (|knows|)(|Mary|) :: S\NP
        : ((LAM X (LAM Y ((KNOW Y) X))) MARY)
C2    (|John|)(|knows| |Mary|) :: S
        : (((LAM X (LAM Y ((KNOW Y) X))) MARY) JOHN)

Final l-command, normal-order evaluated:

    ((KNOW JOHN) MARY) =
    (KNOW JOHN MARY)
```

#-command

```
/\ #

Most likely l-command for the input: (John knows Mary)

  ((KNOW MARY) JOHN) =
  (KNOW MARY JOHN)

Cumulative weight:  55.197394221550994

Most probable analysis for it: (3 1 8)
--------------------------------
ELM  7.54988  (John) :: NP
        : JOHN
ELM  5.61179  (knows) :: (S\NP)/NP
        : (LAM X (LAM Y ((KNOW X) Y)))
ELM  7.75409  (Mary) :: NP
        : MARY
```

```
C1   13.3659  (|knows|)(|Mary|) :: S\NP
        : ((LAM X (LAM Y ((KNOW X) Y))) MARY)
C2   55.1974  (|John|)(|knows| |Mary|) :: S
        : (((LAM X (LAM Y ((KNOW X) Y))) MARY) JOHN)

Final l-command, normal-order evaluated:

    ((KNOW MARY) JOHN) =
    (KNOW MARY JOHN)


Most weighted analysis : (3 1 8)
--------------------------
ELM  7.54988  (John) :: NP
        : JOHN
ELM  5.61179  (knows) :: (S\NP)/NP
        : (LAM X (LAM Y ((KNOW X) Y)))
ELM  7.75409  (Mary) :: NP
        : MARY
C1   13.3659  (|knows|)(|Mary|) :: S\NP
        : ((LAM X (LAM Y ((KNOW X) Y))) MARY)
C2   55.1974  (|John|)(|knows| |Mary|) :: S
        : (((LAM X (LAM Y ((KNOW X) Y))) MARY) JOHN)

Final l-command, normal-order evaluated:

    ((KNOW MARY) JOHN) =
    (KNOW MARY JOHN)
```

=-command

```
/\ = np


Analysis 1
--------------
ELM   (Mary) :: NP
        : CAT

Final l-command, normal-order evaluated:

    CAT =
    (CAT)

Analysis 2
--------------
ELM   (Mary) :: NP
        : MARY

Final l-command, normal-order evaluated:

    MARY =
    (MARY)
```

!-command

```
/\ ! basics-and-features-of-en.txt
          editable files in
 home    : /home/bozsahin/myrepos/thebench/examples/training.demo.basic
          temporary and non-editable files in
 tmp     : /var/tmp/thebench/
 file    :  en.txt (10 entries)
 elements:  10
 s rules :  0 (turned to 0 elements)
 a rules :  0
 basics  :  s np
singleton:
 special :
 features:
 values  :
 POSs    :  v n
also writing to file basics-and-features-of-en.txt.log
```

$-command

```
/\ $ v

Entry: ((KEY 70482) (PARAM 1.0) (PHON knows) (MORPH V)
        (SYN
         ((((BCAT S) (FEATS NIL)) (DIR BS) (MODAL ALL) ((BCAT NP) (FEATS NIL)))
          (DIR FS) (MODAL ALL) ((BCAT NP) (FEATS NIL))))
```

```
        (SEM (LAM X (LAM Y ((KNOW X) Y)))))
--

Entry: ((KEY 38913) (PARAM 1.0) (PHON knows) (MORPH V)
        (SYN
         ((((BCAT S) (FEATS NIL)) (DIR BS) (MODAL ALL) ((BCAT NP) (FEATS NIL)))
          (DIR FS) (MODAL ALL) ((BCAT S) (FEATS NIL))))
        (SEM (LAM S (LAM X ((KNOW S) X)))))
--

Entry: ((KEY 629487) (PARAM 1.0) (PHON knows) (MORPH V)
        (SYN
         ((((BCAT S) (FEATS NIL)) (DIR BS) (MODAL ALL) ((BCAT NP) (FEATS NIL)))
          (DIR FS) (MODAL ALL) ((BCAT NP) (FEATS NIL))))
        (SEM (LAM X (LAM Y ((KNOW Y) X)))))
--

Entry: ((KEY 904879) (PARAM 1.0) (PHON knows) (MORPH V)
        (SYN
         ((((BCAT S) (FEATS NIL)) (DIR BS) (MODAL ALL) ((BCAT NP) (FEATS NIL)))
          (DIR FS) (MODAL ALL) ((BCAT S) (FEATS NIL))))
        (SEM (LAM S (LAM X ((KNOW X) S)))))
--

Entry: ((KEY 445286) (PARAM 1.0) (PHON loves) (MORPH V)
        (SYN
         ((((BCAT S) (FEATS NIL)) (DIR BS) (MODAL ALL) ((BCAT NP) (FEATS NIL)))
          (DIR FS) (MODAL ALL) ((BCAT NP) (FEATS NIL))))
        (SEM (LAM X (LAM Y ((LOVE X) Y)))))
--

Entry: ((KEY 186883) (PARAM 1.0) (PHON loves) (MORPH V)
        (SYN
         ((((BCAT S) (FEATS NIL)) (DIR BS) (MODAL ALL) ((BCAT NP) (FEATS NIL)))
          (DIR FS) (MODAL ALL) ((BCAT NP) (FEATS NIL))))
        (SEM (LAM X (LAM Y ((LOVE Y) X)))))
--
```

The output is in the internal format because this is reported by `bench.lisp`.

`'-'-command`

```
/\ - brings | v :: (s\np[agr=3s])/np : \x\y.bring x y
{ 0: 'el',
  1: {0: 'form', 1: 'brings', 2: 'v'},
  2: { 0: 'cat',
       1: { 0: 'scom',
            1: { 0: 'range',
                 1: { 0: 'range',
                      1: {0: 'basic', 1: 's', 2: []},
                      2: { 0: 'dom',
                           1: {0: 'dir', 1: '\\', 2: '.'},
                           2: {0: 'basic', 1: 'np', 2: [['agr', '3s']]}}},
                 2: { 0: 'dom',
                      1: {0: 'dir', 1: '/', 2: '.'},
                      2: {0: 'basic', 1: 'np', 2: []}}}},
       2: { 0: 'lcom',
            1: { 0: 'lam',
                 1: 'x',
                 2: { 0: 'lam',
                      1: 'y',
                      2: { 0: 'app',
                           1: {0: 'app', 1: 'bring', 2: 'x'},
                           2: 'y'}}}}}}
```

`+`-command

```
/\ + /home/bozsahin/myrepos/thebench/src/bench.user.lisp
/home/bozsahin/myrepos/thebench/src/bench.user.lisp loaded to the processor
```

`>`-command

```
/\ > mysession force
Logging processor output to:  mysession.log
```

`;`-command

```
/\ ;
the contents of /var/tmp/thebench/
total 1.1M
```

```
-rw-r--r-- 1 bozsahin bozsahin 1.8K May 20 11:42 en.txt.src
-rw-r--r-- 1 bozsahin bozsahin  457 May 20 11:42 en.txt.log
-rw-r--r-- 1 bozsahin bozsahin  482 May 20 11:30 nfp.xp.1.2a.1.0c.src.462501.txt.log
-rw-r--r-- 1 bozsahin bozsahin 2.0K May 20 11:30 nfp.xp.1.2a.1.0c.src.462501.txt.src
-rw-r--r-- 1 bozsahin bozsahin 1.9K May 20 10:32 boff.8.0.5a.1.2c.src
-rw-r--r-- 1 bozsahin bozsahin 2.7K May 20 10:32 boff.log
-rw-r--r-- 1 bozsahin bozsahin 2.7K May 20 10:32 nfp.log
-rw-r--r-- 1 bozsahin bozsahin 1.9K May 20 10:32 nfp.xp.1.2a.1.0c.src
-rw-r--r-- 1 bozsahin bozsahin 1.8K May 20 10:32 bon.10.0.5a.1.1c.src
-rw-r--r-- 1 bozsahin bozsahin 2.7K May 20 10:32 bon.log
-rw-r--r-- 1 bozsahin bozsahin  474 May 20 10:32 nohup.out
-rw-r--r-- 1 bozsahin bozsahin  337 May 20 10:32 en.exp.thebench
-rw-r--r-- 1 bozsahin bozsahin   51 May 20 10:32 en.spairs.log
-rw-r--r-- 1 bozsahin bozsahin  500 May 20 10:32 en.spairs.sup
-rw-r--r-- 1 bozsahin bozsahin  18K May 20 10:06 lets-have-a-closer-look
-rw-r--r-- 1 bozsahin bozsahin 3.8K May 16 11:14 en.gram.src
-rw-r--r-- 1 bozsahin bozsahin  458 May 16 11:14 en.gram.log
-rw-r--r-- 1 bozsahin bozsahin 276K May  9 15:22 eve-test.15.1.74a.2.0c.src.170693.txt.src
-rw-r--r-- 1 bozsahin bozsahin  488 May  9 15:22 eve-test.15.1.74a.2.0c.src.170693.txt.log
-rw-r--r-- 1 bozsahin bozsahin 313K May  9 15:19 eve-test.15.1.74a.2.0c.src
-rw-r--r-- 1 bozsahin bozsahin  86K May  9 15:19 eve-test.log
-rw-r--r-- 1 bozsahin bozsahin   52 May  9 15:18 eve.am.sup.log
-rw-r--r-- 1 bozsahin bozsahin 5.6K May  9 15:18 eve.am.sup.sup
-rw-r--r-- 1 bozsahin bozsahin  124 May  9 15:18 eve.exp.thebench
-rw-r--r-- 1 bozsahin bozsahin 275K May  9 15:18 eve.gram.src
-rw-r--r-- 1 bozsahin bozsahin  459 May  9 15:18 eve.gram.log
```

>-command

```
/\ > mysession force
Logging processor output to:  mysession.log
```

/-command

```
/\ /
directory /var/tmp/thebench/ cleared
```

Basic glossary of TheBench

analysis Comparing the synthetic elements of grammar by composition.

auxiliary file A file internally generated by TheBench for the processor, and saved in /var/tmp/thebench. Examples are grammar source as Lisp code and processor-friendly supervision files.

batch mode Use of the '@ command'. You will see TheBench prompt doubled in the log file. TheBench begins to read your commands from a file rather than the terminal. The commands in the command files can themselves be the '@ command; useful for trying many models at batch mode.

editable file A grammar file that you create and save as text, or the grammar files that TheBench generates by 'c command' and 'z command'.

element A (synthetic) element of grammar; one of (12).

grammar, case functions file A file automatically generated by TheBench when you run the 'c command'. File extension is '.sc.arules'. Such files are editable, therefore saved in your working directory. Merging these files with grammar text is up to you. The 'c command' adds them to the currently loaded grammar only.

grammar, text file A textual file containing a grammar with entries in the form of (12). The file extension is optional, and it is up to you. Resides in your working directory.

grammar, re-text file A textual file generated by the system from a '.src' file. The result file is identical in structure to the edited grammar which generated the '.src' file. The system adds a default parameter value and a unique key to identify every entry. Such files are saved in your working directory after fetching the '.src' file from /var/tmp/thebench, indicating that they are editable. The 'z command' does this.

grammar, source code file A textual grammar which is transformed to the processor notation. It is actually Lisp source code. Its extension is '.src' if generated by the system. TheBench saves them in the folder /var/tmp/thebench by default.

identifier, token An atomic element of ASCII symbols with at least one alphabetical symbol, not beginning with one of {+, *, ^, .}, which are the slash modalities. It may contain and begin with: a letter, number, tilde, dash or underscore. Examples are basic categories, feature names and values, relation names, parts of speech, predicate basic terms (names of predicates and arguments), lambda variable names.

identifier, predicate modality An identifier that consists of '_' only. It is a simple convention from Bozşahin (2023), nothing universal, to signify that the stuff before it is the predicate and the stuff right after it is a modality of the predicate rather than an argument. Used in l-command only.

identifier, variable An identifier that begins with '?'. Used in the feature values of basic syntactic categories to represent underspecification.

!identifier, !token An identifier/token in a lambda term prefixed with '!'. The Lisp processor converts such identifiers to a double-quoted string. For example, !_name becomes "_name" in the processor. It is kept for legacy.

intermediate representation Sometimes it is difficult to tell whether a textual element in grammar text has been converted to proper structural representation. Using the 'i command' you can check the internal structure of all entries in a grammar in the form of Python code. The resulting file is editable (e.g. if you want to explore its dictionary data structure in python), therefore saved in your working directory unless specified otherwise. You can do the same check for one element only, without adding it to grammar. Use the '- command' for that.

item Any space-separated sequence of UTF-8 text, for example the phonological material. In representation and processing, it is case-sensitive.

macro file See the batch mode and the '@ command.'

MWE Multiword expression. Something referentially atomic in a surface expression, marked as |..|, for example |the bucket|. Whitespacing is not important. In a grammar entry, multiple words before the part of speech is by definition an MWE. *In a surface expression* they must be within |..| to match that element in analysis.

modality, syntactic One of {^, *, +, .} after a syntactic slash as a further surface constraint. Assumed to be '.' if omitted. Controls the amount (none '*' to all '.') and the degree (harmony '^' disharmony '+') of composition.

part of speech tag An identifier which can be put to various use, e.g. morphological identification, grammatical organization. Such identifiers can be any sequence of ASCII non-space tokens. No universal set is assumed. For example, the 'c command' uses these tags to derive case functions, because the verbs cannot be discerned from their syntactic category alone. Take for example the category (S\NP)\VP: Is this for a verb or an adverb? Somebody has to just designate them as verb or verb-like, to be targeted by the 'c command'.

ranking Looking at the most likely analysis of an expression given a trained grammar.

relational category, asymmetric A category which maps a surface category to another surface category, to indicate that the surface form bearing the first one also bears the second one. In analysis, they apply in the order specified in grammar.

relational category, symmetric A category which maps two surface forms to each other if they are categorially related. In analysis, they are treated as independent elements; whichever takes part in surface structure will be used with its own category. Their order is not important, either in relation specification or in analysis.

singleton A basic category in single or double quotes, in basic and complex categories. For example `(s\np)/"the bucket"` treats `"the bucket"` as a category that can only take on one value, the surface string itself, as in *kicked the bucket* kind of idioms, so that a much narrower reference compared to the more general category `NP` can be discerned; see Bozşahin (2023).

special category Any basic category prefixed with '`@`'. They are application-only, to avoid nested term unification. Useful for coordinands and clitics, i.e. things that can take one complex category no matter what in one fell swoop.

supervision, text file A textual file containing entries in the form of (26). The file extension is optional; it is up to you. Resides in your working directory.

supervision, source code file A supervision file generated by the system from your supervision file text. It has the extension '`.sup`'. It is in the Lisp format that the processor uses. Such files are saved in the folder `/var/tmp/thebench`.

synthesis Putting together of an element of grammar. Although an element has components such as the phonological form, part of speech, the syntactic type and the predicate-argument structure, as far as analytics is concerned, this is a hermetic seal, and only the monad can manipulate the components.

References

Abelson, Harold, Gerald Jay Sussman, and Julie Sussman. 1985. *Structure and Interpretation of Computer Programs*. MIT Press.

Abend, Omri, Tom Kwiatkowski, Nathaniel Smith, Sharon Goldwater, and Mark Steedman. 2017. Bootstrapping language acquisition. *Cognition*, 164:116–143.

Ajdukiewicz, Kazimierz. 1935. Die syntaktische Konnexität. In *Polish Logic 1920–1939*. ed.Storrs McCall, 207–231. Oxford: Oxford University Press. Trans. from *Studia Philosophica*, 1, 1-27.

Bach, Emmon. 1976. An extension of Classical Transformational Grammar. In *Problems in Linguistic Metatheory: Proceedings of the 1976 Conference at Michigan State University*, 183–224. Lansing, MI: Michigan State University.

Baldridge, Jason. 2002. *Lexically Specified Derivational Control in Combinatory Categorial Grammar*. Doctoral Dissertation, University of Edinburgh.

Bar-Hillel, Yehoshua. 1953. A quasi-arithmetical notation for syntactic description. *Language*, 29:47–58.

Bar-Hillel, Yehoshua, Chaim Gaifman, and Eliyahu Shamir. 1960/1964. On categorial and phrase structure grammars. In *Language and Information*. ed.Yehoshua Bar-Hillel, 99–115. Reading, MA: Addison-Wesley.

Bozşahin, Cem. 2023. Referentiality and configurationality in the idiom and the phrasal verb. *Journal of Logic, Language and Information*, 32:175–207.

Bozşahin, Cem. 2024. Mobile sequencers. `arxiv`, 2405.06710v1. Also available in *LingBuzz*.

Bozşahin, Cem. 2025. *Connecting Social Semiotics, Grammaticality and Meaningfulness: The Verb*. Newcastle upon Tyne: Cambridge Scholars.

Brown, Penelope. 1998. Children's first verbs in Tzeltal: Evidence for an early verb category. *Linguistics*, 36:713–753.

Brown, Roger. 1973. *A First Language: the Early Stages*. Cambridge, MA: Harvard University Press.

Cabay, S, and LW Jackson. 1976. A polynomial extrapolation method for finding limits and antilimits of vector sequences. *SIAM Journal on Numerical Analysis*, 13:734–752.

Crain, Stephen, and Rosalind Thornton. 1998. *Investigations in Universal Grammar*. Cambridge, MA: MIT Press.

Eisner, Jason. 1996. Efficient normal-form parsing for Combinatory Categorial Grammar. In *Proceedings of the 34th Annual Meeting of the ACL*, 79–86.

Ellis, Willis D. 1938. *A Source Book of Gestalt Psychology*. London: Routledge and Kegan Paul.

Gallier, Jean. 2011. *Discrete Mathematics*. Springer.

Gentner, Dedre. 1982. Why nouns are learned before verbs: Linguistic relativity versus natural partitioning. In *Language Development, vol.2: Language, Thought and Culture*. ed.Stan A. Kuczaj II, 301–334. Hillsdale, New Jersey: Lawrence Erlbaum.

Grimshaw, Jane. 1990. *Argument Structure*. Cambridge, MA: MIT Press.

Hale, Ken, and Samuel Jay Keyser. 2002. *Prolegomenon to a Theory of Argument Structure*. Cambridge, MA: MIT Press.

Hale, Kenneth. 1983. Warlpiri and the grammar of non-configurational languages. *Natural Language and Linguistic Theory*, 1:5–47.

Hoeksema, Jack, and Richard D. Janda. 1988. Implications of process-morphology for Categorial Grammar. In *Categorial Grammars and Natural Language Structures*. eds.Richard T. Oehrle, Emmon Bach, and Deirdre Wheeler. Dordrecht: D. Reidel.

Klein, Ewan, and Ivan Sag. 1985. Type-driven translation. *Linguistics and Philosophy*, 8:163–201.

Klein, Wolfgang. 1994. *Time in Language*. London: Routledge.

Klein, Wolfgang, Ping Li, and Henriette Hendriks. 2000. Aspect and assertion in Mandarin Chinese. *Natural Language and Linguistic Theory*, 18:723–770.

Koffka, Kurt. 1936. *Principles of Gestalt Psychology*. London: Kegan Paul.

Lambek, Joachim. 1958. The mathematics of sentence structure. *American Mathematical Monthly*, 65:154–170.

Mac Lane, Saunders. 1971. *Categories for the Working Mathematician*. Berlin/New York: Springer.

MacWhinney, B. 2000. *The CHILDES project: Tools for analyzing talk. Third edition*. Mahwah,

NJ: Lawrence Erlbaum Associates.

Moggi, Eugenio. 1988. *Computational Lambda-Calculus and Monads*. LFCS, University of Edinburgh.

Montague, Richard. 1970. English as a formal language. In *Linguaggi nella Società e nella Technica*. ed.Bruno Visentini, 189–224. Milan: Edizioni di Communità. Reprinted as Thomason 1974:188-221.

Montague, Richard. 1973. The proper treatment of quantification in ordinary English. In *Approaches to Natural Language*. eds.J. Hintikka and P. Suppes. Dordrecht: D. Reidel.

Ross, John Robert. 1967. *Constraints on Variables in Syntax*. Doctoral Dissertation, MIT. Published as *Infinite Syntax!*, Ablex, Norton, NJ, 1986.

Sapir, Edward. 1924/1949. The grammarian and his language. In *Selected Writings of Edward Sapir in Language, Culture, and Personality*. ed.David G. Mandelbaum. Berkeley: University of California Press. Originally published in *American Mercury 1: (1924) 149–155*.

Schmerling, Susan. 1981. The proper treatment of the relationship between syntax and phonology. Paper presented at the 55th annual meeting of the LSA, San Antonio TX.

Schmerling, Susan. 2018. *Sound and Grammar: a Neo-Sapirian Theory of Language*. Leiden/Boston: Brill.

Schönfinkel, Moses Ilyich. 1920/1924. On the building blocks of mathematical logic. In *From Frege to Gödel*. ed.Jan van Heijenoort. Harvard University Press, 1967. Prepared first for publication by H. Behmann in 1924.

Steedman, Mark. 2000. *The Syntactic Process*. Cambridge, MA: MIT Press.

Steedman, Mark. 2020. A formal universal of natural language grammar. *Language*, 96:618–660.

Steedman, Mark, and Jason Baldridge. 2011. Combinatory Categorial Grammar. In *Nontransformational Syntax*. eds.R. Borsley and Kersti Börjars, 181–224. Oxford: Blackwell.

Swadesh, Morris. 1938. Nootka internal syntax. *International Journal of American Linguistics*, 9:77–102.

ed.Thomason, Richmond. 1974. *Formal Philosophy: Papers of Richard Montague*. New Haven, CT: Yale University Press.

Tomasello, Michael. 1992. *First Verbs: a Case Study in Early Grammatical Development*. Cambridge: Cambridge University Press.

Wertheimer, Max. 1924. Gestalt theory. English translation published in Ellis (1938).

Williams, Edwin. 1994. *Thematic Structure in Syntax*. Cambridge, MA: MIT Press.

Wojdak, Rachel. 2000. Nuu-chah-nulth modification: Syntactic evidence against category neutrality. In *Papers for the 35th International Conference on Salish and Neighbouring Languages (UBCWPL)*, vol. 3, 269–281.

Woolford, Ellen. 2006. Lexical case, inherent case, and argument structure. *Linguistic Inquiry*, 37:111–130.

Zettlemoyer, Luke, and Michael Collins. 2005. Learning to map sentences to logical form: Structured classification with probabilistic categorial grammars. In *Proc. of the 21st Conf. on Uncertainty in Artificial Intelligence*. Edinburgh.

Table 1: The command interface of `TheBench`. There is also the phantom command called '`pass`', which is useful for commenting on the tasks in the '`@ command`'s batch processing files. For example, '`pass the next command generates case functions`' does nothing but echoes itself. Any command in the interface can be put in the batch files as long as it can be processed without online input.

```
________  Letter commands are processor commands; symbol commands are for display or setup
________  Dots are referred in sequence; .? means optional; .* means space-separated items
 a .*    | analyzes . in the current grammar; MWEs must be enclosed in |, e.g. |the bucket|
 c .*    | case functions generated for current grammar from elements with POSs .
 e .*    | evaluates the python expression . at your own risk (be careful with deletes)
 g .     | grammar text .  checked and its source made current (.src file goes to /var/tmp/thebench/)
 i .     | intermediate representation of current grammar saved (file . goes to /var/tmp/thebench/)
 k       | reports categorial skeleton of the current grammar---its distinct syntactic categories
 l ..?   | Lisp function . is called with args ., which takes them as strings
 o .*    | OS/shell command . is run at your own risk (be careful with deletes)
 r .*    | ranks . in the current grammar; MWEs must be enclosed in |, e.g. |the bucket|
 t ...   | trains grammar in file . on data in file . using training parameters in file .
 z .     | source . located in /var/tmp/thebench/ and saved as editable grammar locally (.txt)
 @ ..    | does (nested) commands in file . (1 command/line, 1 line/command); forces output to .log
 , .*?   | displays analyses for solutions numbered ., all if none provided
 # .?    | displays ranked analyses; outputs only [string likeliest-solution] pair if . is 'bare'
 = .*    | displays analyses onto basic cats in .
 ! .?    | shows basic cats and features of current grammar (optionally saves to file .log)
 $ .*    | shows the elements with parts of speech in .
 - .     | shows (without adding) the intermediate representation of element .
 + .     | processor adds Lisp code in file .
 > ..?   | logs processor output to file .log; if second . is 'force' overwrites if exists
 <       | logging turned off
 ;       | displays the contents of the /var/tmp/thebench/ directory
 /       | clears the /var/tmp/thebench/ directory
 ?       | displays help
________| Use UP and DOWN keys for command recall from use history
```

Table 2: Processor functions accessible from the experiment files and the command line.

`beam-on`	Turns the beam on. For training.
`beam-off`	Turns the beam off (default). For training.
`beam-value`	Shows current properties of the beam processor (status and exponent). For display.
`lambda-on`	Turns on display of lambda terms at every step of analysis (default). For display.
`lambda-off`	Turns it off. Final results are always shown. For display.
`monad-all`	Processor set to use all monad rules (default). For analysis and training.
`monad-montague`	Processor set to use only application in composition (**A** and **T**). For analysis and training.
`nfparse-on`	Turns normal-form parsing on (default). For display.
`nfparse-off`	Turns normal-form parsing off. For analysis and training.
`onoff`	List of on/off switches that control analysis.
`oov-on`	Turns on out-of-vocabulary treatment. Two dummy entries assumed for OOV items: One with the category `@X\@X` and the other with `@X/@X`. For analysis and training.
`oov-off`	Turns off out-of-vocabulary treatment (default). For analysis and training.
`show-config`	Shows current values of all the properties above.